

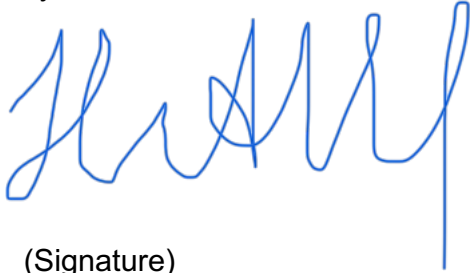
A Comparative Study of Transformer and xLSTM Policy Networks for Molecular Generation

Scientific work to obtain the degree
Master of Science (M.Sc.)
at TUM Campus Straubing for Biotechnology and Sustainability of the Technical University of Munich.

Examiner	Prof. Dr. Dominik Grimm Professorship for Bioinformatics
Supervisor	Prof. Dr. Dominik Grimm Professorship for Bioinformatics
Submitted by	Nikita Ivanov Matrikel-Nr. 03795558
Submitted at	22.05.2026, Straubing

Affidavit

I hereby certify that this master's thesis was written without the help of third parties and only with the sources and aids indicated. All passages used have been identified. This thesis has not yet been submitted in the same or a similar form to any examination authority.

A handwritten signature in blue ink, appearing to be 'Nikita Ivanov', written in a cursive style.

(Signature)

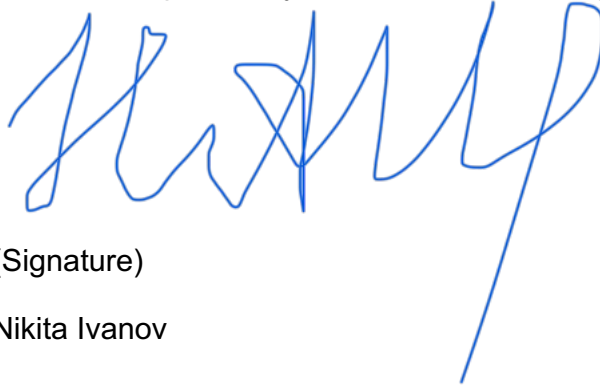
Nikita Ivanov

Straubing, 22.05.2026

AI-Use Statement

During the preparation of this thesis, the author used ChatGPT to improve grammar and language clarity.

After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

A handwritten signature in blue ink, appearing to be 'Nikita Ivanov', written in a cursive style.

(Signature)

Nikita Ivanov

Straubing, 22.05.2026

Abstract

This thesis investigates the suitability of transformer-based and xLSTM-based architectures as policy networks within the GraphXform framework for reinforcement learning-based molecular generation. The study extends the existing workflow by integrating xLSTM-based models and conducting a systematic empirical comparison with transformer-based approaches. The evaluation covers molecular generation performance on GuacaMol benchmark tasks, the exploration–exploitation balance under different fine-tuning and beam search configurations, and computational efficiency in terms of runtime and GPU memory consumption. The results show that both architectures achieve comparable performance on medium and hard benchmark tasks, while transformer-based models demonstrate a slight overall advantage and faster generation speed. In contrast, xLSTM-based models exhibit substantially lower memory consumption and better scalability under large beam widths. Furthermore, the experiments highlight the strong influence of stochasticity and the task-dependent nature of hyperparameter effects. Overall, transformer-based models appear more suitable for the considered workflow, while xLSTM offers advantages in memory efficiency and scalability.

Contents

Affidavit	ii
AI-Use Statement	iii
Abstract	iv
List of Figures	vii
List of Tables	ix
Formula symbols and abbreviations	xi
1. Introduction	1
1.1. Background and Related Work	1
1.2. Motivation	5
1.3. Research Objectives	6
1.4. Contributions	6
2. Methodology	7
2.1. General Workflow	7
2.2. Molecular Graph	8
2.3. Sequential Molecular Graph Design	11
2.4. Molecular Optimization	12
2.5. Learning Algorithm	13
2.6. Policy Network Architecture	17
2.7. Network Hyperparameters	31
2.8. Experimental Setup and Evaluation Protocol	32
2.9. Runtime and Computational Budget	35
3. Results and Discussion	37
3.1. Pretraining Dynamics and Convergence Behavior	37
3.2. Evaluation of Model Performance under the Baseline Configuration	39
3.3. Impact of Fine-tuning under a Fixed Beam Width	41
3.4. Impact of the Joint Effect of Fine-tuning and Beam Width Tuning	43
3.5. Computational Efficiency During Pretraining	48
3.6. Computational Efficiency During Generation	49
3.7. Effect of Beam Width on Computational Efficiency	53

4. Conclusion	56
Bibliography	58
Appendix A. Appendix	62

List of Figures

1.1. Taxonomy of molecular generation methods by representation and modeling paradigm	2
2.1. Reinforcement learning framework for molecular generation, illustrating the interaction between the agent, environment, and objective function	8
2.2. Representation of a molecule as a hydrogen-suppressed graph	9
2.3. Example of sequence decoding using the TASAR algorithm with beam width $\beta = 3$ and step size $\sigma = 2$	15
2.4. Decomposition of a molecule generation action into a sequence of sub-actions .	19
2.5. Node embedding and feature augmentation	20
2.6. Transformer-based node representation model (single transformer block)	21
2.7. xLSTM-based node representation model	23
2.8. Graph neural network encoder layer	24
2.9. Example of the message passing process in the proposed GNN	25
2.10. Single xLSTM block	27
2.11. Single mLSTM cell	28
2.12. Action-level probability distribution	30
3.1. Training and validation loss curves for transformer-based and xLSTM-based models under different batch size configurations during pretraining	38
3.2. Baseline performance of transformer-based and xLSTM-based models	39
3.3. Impact of fine-tuning intensity on transformer-based and xLSTM-based models ($\beta = 512$)	41
3.4. Joint effect of fine-tuning intensity and beam width on model performance for hard tasks	44
3.5. Mean total score and standard deviation across all configurations for transformer-based and xLSTM-based models on hard tasks. Error bars indicate one standard deviation around the mean	45
3.6. GPU memory usage over time for transformer-based and xLSTM-based models under different batch size configurations	49
3.7. Total number of optimization epochs completed within an 8-hour time budget as a function of fine-tuning batch size for transformer-based and xLSTM-based models	50
3.8. GPU memory usage over time during molecule generation for transformer-based and xLSTM-based models	52

3.9. Total number of optimization epochs completed within a fixed 8-hour time budget as a function of beam width for transformer-based and xLSTM-based models . .	54
3.10. Peak GPU memory usage as a function of beam width for transformer-based and xLSTM-based models. OOM denotes out-of-memory error	54

List of Tables

2.1. Overview of experimental configurations used to evaluate model performance. Baseline – evaluation under the default configuration ($\beta = 512$, $\sigma = 12$, 20 fine-tuning batches); FT – analysis of the impact of fine-tuning under a fixed beam width ($\beta = 512$); Joint – investigation of the joint effect of fine-tuning and beam width tuning.	35
2.2. Overview of experimental configurations and computational budget. Baseline – evaluation under the default configuration ($\beta = 512$, $\sigma = 12$, 20 fine-tuning batches); FT – analysis of the impact of fine-tuning under a fixed beam width ($\beta = 512$); Joint – investigation of the joint effect of fine-tuning and beam width tuning.	36
3.1. Empirical grouping of GuacaMol benchmark tasks by difficulty under the baseline configuration.	40
3.2. Performance comparison on hard tasks between the proposed models and baseline methods from the GuacaMol [20]	47
3.3. Computational efficiency during pretraining for transformer-based and xLSTM-based models.	48
3.4. Mean and standard deviation of the total number of completed optimization epochs within an 8-hour time budget for different model architectures and batch size configurations.	51
A.1. Empirical task categorization based on performance under the baseline configuration. Bold values indicate the highest score for each task across all model configurations.	62
A.2. Performance on medium and hard tasks under varying fine-tuning intensities with a fixed beam width ($\beta = 512$). Results are reported for all model configurations across different numbers of fine-tuning batches per epoch. T512 – transformer pretrained with batch size 512; T256 – transformer pretrained with batch size 256; L512 – xLSTM pretrained with batch size 512; L256 – xLSTM pretrained with batch size 256. Bold values indicate the highest score for a given task within a specific fine-tuning configuration, while highlighted cells denote the best result achieved across all configurations for that task.	63

A.3. Comparison of model performance under fixed configuration and oracle-like selection. T512 – transformer pretrained with batch size 512; T256 – transformer pretrained with batch size 256; L512 – xLSTM pretrained with batch size 512; L256 – xLSTM pretrained with batch size 256. Bold values indicate the highest score for a given task within a specific selection setup, while highlighted cells denote the best result achieved across all setups for that task.	64
A.4. Performance on hard tasks as a function of fine-tuning intensity ($\beta = 512$). Bold values indicate the highest score for a given task within a specific fine-tuning configuration, while highlighted cells denote the best result achieved across all configurations for that task.	65
A.5. Performance on hard tasks as a function of fine-tuning intensity ($\beta = 1024$). Bold values indicate the highest score for a given task within a specific fine-tuning configuration, while highlighted cells denote the best result achieved across all configurations for that task.	65
A.6. Performance on hard tasks as a function of fine-tuning intensity ($\beta = 2048$). Bold values indicate the highest score for a given task within a specific fine-tuning configuration, while highlighted cells denote the best result achieved across all configurations for that task.	65
A.7. Number of training epochs completed within an 8-hour time budget across different fine-tuning configurations and fixed beam widths ($\beta = 512$). T512 – transformer pretrained with batch size 512; T256 – transformer pretrained with batch size 256; L512 – xLSTM pretrained with batch size 512; L256 – xLSTM pretrained with batch size 256.	66
A.8. Number of training epochs completed within an 8-hour time budget as a function of fine-tuning intensity ($\beta = 512$).	67
A.9. Number of training epochs completed within an 8-hour time budget as a function of fine-tuning intensity ($\beta = 1024$).	67
A.10. Number of training epochs completed within an 8-hour time budget as a function of fine-tuning intensity ($\beta = 2048$).	67

Formula symbols and abbreviations

Symbol	Description	Unit
$\mathbf{a}$	Atom sequence of a molecular graph	(-)
a_i	Atom at index i in the atom sequence	(-)
a_0	Virtual atom / virtual node	(-)
$\hat{a}_i$	Latent embedding of atom i	(-)
A_{ij}	Attention score between nodes i and j	(-)
$\mathbf{B}$	Bond matrix of a molecular graph	(-)
b_{ij}	Learnable attention bias between nodes i and j	(-)
$\mathbf{C}_t$	mLSTM matrix memory state at step t	(-)
d	Latent representation dimension	(-)
$\mathbf{e}_{ij}$	Embedding of the bond between nodes i and j	(-)
$\mathbf{e}_t$	Input embedding at sequence position t	(-)
E	Set of graph edges	(-)
f	Objective function assigning a score to a molecule	(-)
f_t	Forget gate of the mLSTM cell	(-)
g_0	Graph-level projection function for action level 0	(-)
g_2	Graph-level projection function for action level 2	(-)
G	Molecular graph defined by nodes and edges	(-)
$\mathbf{h}_i^{(l)}$	Representation of node i at GNN layer l	(-)
$\mathbf{H}^{\text{GNN}}$	Node representations produced by the GNN encoder	(-)
$\mathbf{H}^{\text{xLSTM}}$	Node representations produced by the xLSTM model	(-)
i_t	Input gate of the mLSTM cell	(-)
k	Number of atom types in the atom alphabet	(-)
$\mathbf{k}_t$	Key representation in the mLSTM cell	(-)
L	Number of GNN layers	(-)
m	Molecule / molecular graph	(-)
m^*	Optimal molecule maximizing the objective function	(-)
$\mathbf{m}_{j \rightarrow i}^{(l)}$	Message passed from node j to node i at GNN layer l	(-)
m_t	Stabilizer state of the mLSTM cell	(-)
n	Number of real atoms in a molecule	(-)
N_{best}	Number of top-performing molecules retained in BESTFOUND	(-)
$\mathbf{n}_t$	Normalizer state of the mLSTM cell	(-)
$\mathcal{N}(i)$	Set of neighboring nodes of node i	(-)
o	Bond order selected during molecule generation	(-)

Symbol	Description	Unit
$p_j^{(0)}$	Probability of selecting atom type Σ_j at action level 0	(-)
$p_j^{(2)}$	Probability of selecting bond order j at action level 2	(-)
$P^{(0)}$	Action-level 0 distribution	(-)
$P^{(1)}$	Action-level 1 distribution	(-)
$P^{(2)}$	Action-level 2 distribution	(-)
$\mathbf{q}_t$	Query representation in the mLSTM cell	(-)
$\mathbf{r}$	Embedding of the current action level	(-)
t	Generation or recurrent time step	(-)
T	Terminal time step of the generation trajectory	(-)
$\mathbf{v}_t$	Value representation in the mLSTM cell	(-)
V	Set of graph nodes	(-)
$\mathbf{w}_i^{(0)}$	Embedding indicating whether atom i was selected at action level 0	(-)
$\mathbf{w}_i^{(1)}$	Embedding indicating whether atom i was selected at action level 1	(-)
$\mathbf{W}_m$	Learnable message-passing weight matrix	(-)
$\mathbf{W}_o$	Learnable feed-forward projection matrix	(-)
$x^{(t)}$	Action selected at step t	(-)
y	Maximum bond order	(-)
$\mathbf{z}_i$	Embedding of the degree of atom i	(-)
Greek characters		
Σ	Atom alphabet	(-)
β	Beam width	(-)
σ	Step size in the TASAR algorithm	(-)
θ	Parameters of the policy network	(-)
$\zeta_i^{(0)}$	Probability of selecting node i at action level 0	(-)
$\zeta_i^{(1)}$	Probability of selecting node i at action level 1	(-)
ψ_0	Node-level projection function for action level 0	(-)
ψ_1	Node-level projection function for action level 1	(-)
ϕ	Non-linear activation function used in the GNN encoder	(-)
π_θ	Policy parameterized by θ	(-)
Sets		
$\mathcal{A}$	Action space	(-)
$\mathcal{M}$	Space of valid molecules	(-)
Abbreviations		
ADDATOM	Add atom action	(-)
ADDBOND	Add bond action	(-)
CEM	Cross-Entropy Method	(-)
ChEMBL	Chemical database of bioactive molecules	(-)

Symbol	Description	Unit
CUDA	Compute Unified Device Architecture	(-)
cuDNN	CUDA Deep Neural Network library	(-)
DONTCHANGE	Termination action	(-)
FT	Fine-tuning	(-)
GEGL	Genetic Expert-Guided Learning	(-)
GNN	Graph Neural Network	(-)
GPU	Graphics Processing Unit	(-)
JK	Jumping Knowledge	(-)
LSTM	Long Short-Term Memory	(-)
MCTS	Monte Carlo Tree Search	(-)
MDP	Markov Decision Process	(-)
mLSTM	Matrix Long Short-Term Memory	(-)
MPO	Multi-Property Optimization	(-)
OOM	Out-of-memory	(-)
ReLU	Rectified Linear Unit	(-)
RL	Reinforcement Learning	(-)
SELFIES	Self-Referencing Embedded Strings	(-)
SIL	Self-Imitation Learning	(-)
SMILES	Simplified Molecular Input Line Entry System	(-)
TASAR	Take a Step and Reconsider	(-)
TPSA	Topological Polar Surface Area	(-)
xLSTM	Extended Long Short-Term Memory	(-)

1. Introduction

1.1. Background and Related Work

Molecular design plays a crucial role in a wide range of applications, including drug discovery, materials science, and chemical engineering. The size of the chemical search space, estimated to contain between 10^{60} and 10^{100} possible molecules [1], makes exhaustive exploration infeasible and renders traditional manual design approaches both time-consuming and resource-intensive. As a result, generative modeling techniques have emerged as a promising tool for discovering novel molecules with desired properties [2, 3].

Molecular generation methods can be broadly categorized along two dimensions: the choice of molecular representation and the underlying generation strategy. Molecules may be represented either as strings (e.g., SMILES [4] or SELFIES [5]) or as graphs that explicitly encode atomic structure [6, 7, 8], while generation strategies range from classical search-based methods [1, 9], to learning-based approaches relying on neural networks [3, 2]. This perspective defines four main categories: string-based search methods, string-based learning methods, graph-based search methods, and graph-based learning methods (**Figure 1.1**).

String-based representations, such as SMILES, provide a compact and linear encoding of molecular structures, making them well-suited for sequence modeling with recurrent neural networks and transformers [10, 11]. This enables efficient pretraining on large molecular datasets and straightforward integration with deep learning frameworks. However, these representations are non-unique and sensitive to syntax, which can lead to invalid molecules and complicate reinforcement learning-based optimization [12, 13]. In addition, their sequential nature makes it difficult to explicitly capture underlying graph structure and enforce chemical constraints during generation [14, 6].

Graph-based representations address these limitations by modeling molecules as explicit graph structures, where atoms correspond to nodes and bonds to edges [7, 6]. This allows for more natural incorporation of chemical constraints and generally improves the validity and controllability of generated molecules. At the same time, graph-based approaches introduce higher modeling complexity, as molecule generation involves sequential graph construction with a large and structured action space [6, 7]. As a result, such methods typically require more sophisticated architectures and incur higher computational costs compared to string-based approaches [2].

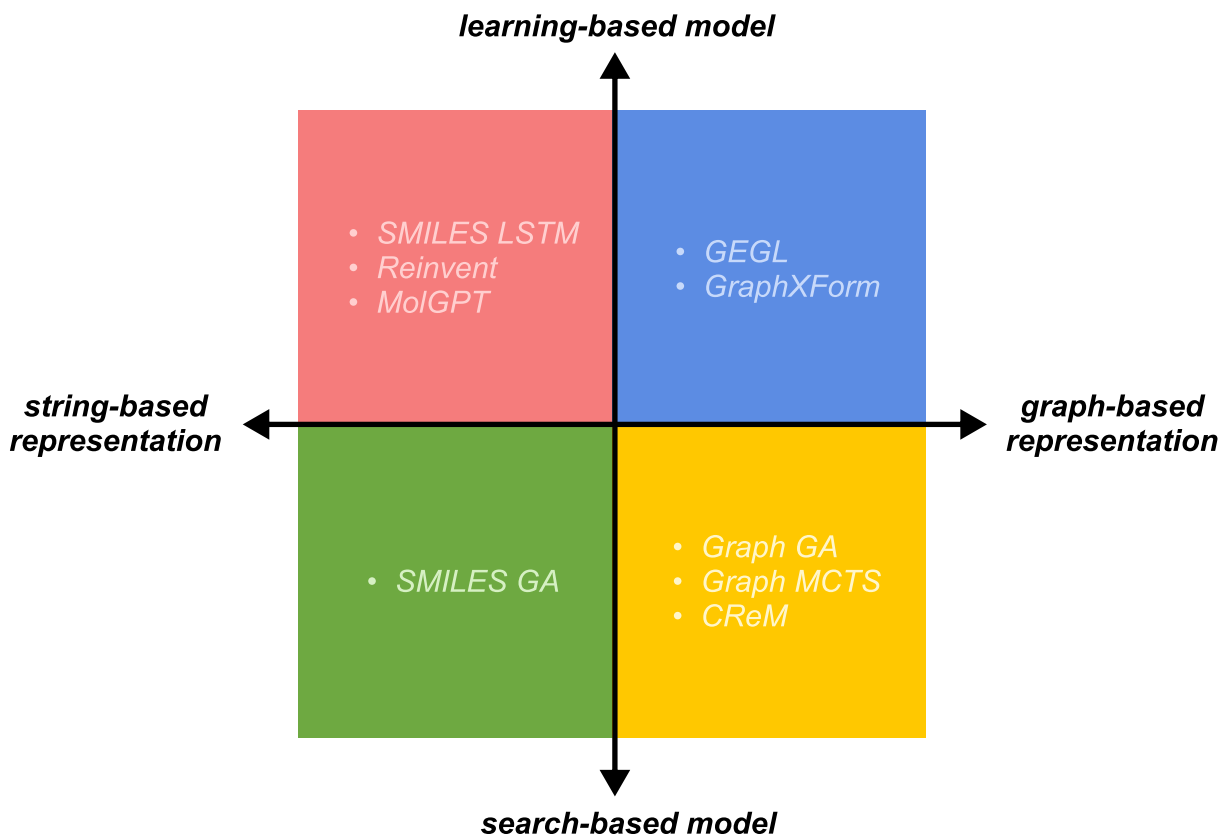


Figure 1.1.: Taxonomy of molecular generation methods by representation and modeling paradigm

From the perspective of generation strategy, search-based methods explore the chemical space through heuristic procedures such as mutation, crossover, or tree expansion [1]. These approaches are flexible and can incorporate domain knowledge directly, but they tend to suffer from low sample efficiency and limited generalization, as each candidate molecule must be evaluated independently [2].

In contrast, learning-based methods train neural models to capture the distribution of chemical structures and generate new candidates accordingly [3, 2]. Leveraging prior knowledge from data improves sample efficiency and enables generalization across tasks, but these methods require substantial training data and computational resources, and their performance can be sensitive to model design and hyperparameters.

To better illustrate how these different representations and generation strategies are realized in practice, we next review a set of representative molecular generation methods from each category. These examples highlight the diversity of existing approaches and provide context for the methodological choices considered in this work.

We begin with string-based search methods. One representative example is the SMILES-based genetic algorithm ChemGE [15], which treats molecular generation as a population-

based optimization problem, where candidate molecules represented as strings are iteratively modified through mutation and selection operations guided by a fitness function. This approach was shown to efficiently explore the chemical space in parallel, generating large and diverse sets of molecules, including hundreds of candidates with better predicted binding affinity than known compounds and outperforming several deep learning-based methods in terms of search efficiency under comparable computational resources.

Continuing with graph-based search methods, graph-based genetic algorithms operate directly on molecular graphs, applying mutation and recombination operations to optimize molecular structures while preserving chemical validity. These methods have demonstrated [16] competitive and often superior performance compared to machine learning approaches for property optimization, achieving higher objective scores while being several orders of magnitude more computationally efficient and capable of exploring distant regions of the chemical space within a limited number of generations.

Another graph-based search approach is Monte Carlo Tree Search (MCTS), which formulates molecule generation as a sequential decision process over graph expansions, where candidate structures are explored through a tree-based search guided by a scoring function [7]. In a comparative study [16], MCTS-based approaches also achieved results comparable to recurrent neural network models, although their performance was found to be more sensitive to the composition of the training data and generally less robust than genetic algorithm-based methods.

Finally, a more advanced representative of this group is CReM (Chemically Reasonable Mutations) [17], a fragment-based molecular generation approach that constructs new molecules by replacing substructures with chemically valid fragments derived from a predefined database, ensuring structural validity by design. Unlike purely data-driven models, CReM does not rely on a learned generative model but instead performs guided search through chemically meaningful transformations. It has been shown to generate diverse and synthetically feasible compounds while allowing flexible control over properties such as novelty and complexity, although its expressiveness is limited by the coverage of the fragment library and its inability to introduce entirely new structural motifs.

We now turn to string-based learning approaches. A fundamental example of this category is the SMILES LSTM model [10], which represents molecules as sequences and employs recurrent neural networks with long short-term memory (LSTM) units to learn a statistical “chemical language”, enabling the generation of novel molecular structures by autoregressively predicting SMILES tokens. In the original study, such models were shown to generate molecules with physicochemical properties closely matching the training data and, after fine-tuning on small sets of active compounds, were able to reproduce up to 14% and 28% of held-out molecules for antibacterial and antimalarial targets, respectively, demonstrating their effectiveness for de novo drug design.

A further extension of the SMILES LSTM model is represented by REINVENT [9], which incorporates reinforcement learning into the generative framework. In this approach, a pretrained

SMILES-based LSTM model is fine-tuned using augmented episodic likelihood to optimize molecular properties according to a task-specific reward function. This enables targeted molecule generation, and in the original study the model was able to generate compounds with desired characteristics across multiple tasks, including producing over 95% of molecules predicted to be active against a biological target and recovering active compounds not present in the training data.

Another representative of this category, based on a different family of generative models, is MolGPT [18]. It applies transformer-based architectures to molecular generation by modeling SMILES sequences using self-attention mechanisms, enabling the capture of long-range dependencies and more expressive sequence representations. In extended formulations such as MolRL-MGPT, transformer-based models are further integrated with reinforcement learning in a multi-agent framework, where multiple generators collaboratively explore different regions of the chemical space; this approach has demonstrated at the time of publication strong performance on the GuacaMol benchmark and has been successfully applied to the design of candidate inhibitors for SARS-CoV-2 protein targets.

Finally, we consider graph-based learning approaches. A representative method in this category is GEGL (Genetic Expert-Guided Learning) [19], which combines deep neural networks with genetic algorithms by using expert-designed mutation operators to generate high-quality training targets for imitation learning. In this framework, a neural generative model is iteratively improved using candidates produced by a genetic “expert”, enabling efficient exploration of the chemical space. This approach achieved state-of-the-art performance at the time of publication, outperforming previous methods across a wide range of tasks, including achieving the highest scores on 19 out of 20 benchmarks in GuacaMol [20] and significantly improving optimization metrics such as penalized logP.

However, a recently proposed workflow, GraphXform [21], further improves upon these results. At the time of publication, it demonstrated state-of-the-art performance, outperforming existing molecular design approaches on the GuacaMol benchmark as well as on additional solvent design tasks, while also enabling flexible incorporation of structural constraints and initialization from existing molecules. These results highlight the effectiveness of combining graph representations with transformer-based policies for molecular optimization.

GraphXform formulates molecular generation as a sequential decision-making process over molecular graphs, where atoms and bonds are iteratively added to construct valid structures. The model employs a decoder-only graph transformer architecture, combining the expressive power of self-attention with graph representations, and is trained using a paradigm of pretraining followed by reinforcement learning-based fine-tuning with a self-improvement mechanism.

1.2. Motivation

Despite the strong performance of GraphXform, its effectiveness largely depends on the choice of the policy network, which plays a central role in guiding the molecular generation process. In the current formulation, this role is fulfilled by a transformer-based architecture, which offers high expressiveness but comes with notable computational costs, including quadratic complexity with respect to sequence length and increased memory consumption [22, 23, 24].

This raises the question of whether alternative architectures could provide a more favorable balance between modeling capacity and computational efficiency.

Recently, xLSTM has been introduced [25] as a modern extension of classical LSTM architecture [26, 27], aimed at improving scalability and performance in sequence modeling tasks. The model incorporates architectural enhancements such as exponential gating and modified memory structures, enabling more efficient information flow and improved parallelization compared to traditional recurrent networks. As a result, xLSTM achieves performance competitive with state-of-the-art transformer and state-space models while retaining the favorable computational properties of recurrent architectures, including more efficient scaling with sequence length. These characteristics make xLSTM a promising alternative for applications where both modeling capacity and computational efficiency are critical.

Moreover, xLSTM-based architectures have already demonstrated strong performance in biological and chemical domains. Recent work on Bio-xLSTM shows that such models can effectively generate and model molecular sequences, learn rich representations, and perform in-context learning for small molecules, while achieving competitive results compared to transformer-based approaches [28]. In parallel, a graph-based extension MolGraph-xLSTM has demonstrated consistent improvements in molecular property prediction tasks, achieving performance gains across multiple benchmark datasets and effectively capturing long-range dependencies in molecular graphs [29]. These results suggest that xLSTM is not only a theoretically appealing alternative, but also a practically effective architecture for modeling molecular data.

Given these properties, it is natural to consider whether xLSTM could serve as a viable alternative to transformer-based policy networks within the GraphXform framework. While transformers have demonstrated strong performance in this setting, they are associated with high computational and memory requirements. In contrast, xLSTM offers more favorable scaling characteristics and has recently shown promising results in molecular modeling tasks. However, a systematic comparison of these architectures in the context of molecular generation and reinforcement learning-based optimization has not yet been conducted.

1.3. Research Objectives

The aim of this work is to compare transformer-based and xLSTM-based architectures and evaluate their suitability as policy networks within the GraphXform workflow.

To achieve this goal, the following objectives are defined:

- Pretrain four models corresponding to the considered architectures and configurations
- Compare model performance under a baseline configuration using the GuacaMol benchmark
- Analyze the exploration–exploitation balance under fixed exploration hyperparameters by varying the number of fine-tuning batches per epoch while keeping the beam width fixed
- Analyze the joint effect of fine-tuning and beam width on the exploration–exploitation balance by varying the number of fine-tuning batches per epoch and the beam width simultaneously
- Evaluate the computational efficiency of both architectures

1.4. Contributions

In this work, we build upon the existing GraphXform framework and prior studies on transformer-based molecular generation. The main contributions of this thesis are as follows:

- Implementation and pretraining of xLSTM-based models for molecular generation
- Integration of xLSTM-based policy networks into the GraphXform workflow
- Comprehensive empirical comparison between transformer-based and xLSTM-based architectures in terms of molecular generation performance and computational efficiency
- Analysis of the impact of key hyperparameters on the exploration–exploitation balance
- Investigation of the role of stochasticity in reinforcement learning-based molecular optimization

2. Methodology

2.1. General Workflow

The general experimental setup is based on the GraphXform framework, which serves as the baseline for this study. While the original implementation employs transformer-based policy networks, in this work we extend the framework by introducing xLSTM-based alternatives. The core components of the workflow, including the reinforcement learning procedure and evaluation protocol, are adopted from prior work, while the xLSTM-based model architecture, its training configurations, and the comparative analysis of transformer-based and xLSTM-based models are developed as part of this thesis.

Within this framework, the molecular generation process can be formulated as a reinforcement learning problem, where the goal is to construct molecules that maximize a predefined objective function. This setting consists of several key elements: an agent, an environment, and an objective function (**Figure 2.1**) [30].

In the presented framework, the *agent* is implemented as a neural network that parameterizes the *policy*, specifically either a transformer or an xLSTM-based model. However, the formulation itself is not restricted to these architectures. More generally, the agent can be any method capable of taking a molecular graph representation as input and mapping it to a probability distribution over the action space. In this sense, the framework is model-agnostic and can, in principle, be combined with alternative approaches, such as evolutionary or heuristic optimization methods.

This abstraction highlights that the proposed framework defines the generation process independently of the specific choice of model, allowing different classes of algorithms to be used and compared within the same setup.

The *environment* is represented by the *molecular graph*, while the agent iteratively modifies this graph through a sequence of actions.

The *action space* $\mathcal{A}$ consists of graph transformations, including adding atoms, creating bonds, or terminating the generation process. Applying an action $x^{(t)} \in \mathcal{A}$ updates the molecular graph $m^{(t)}$, resulting in a new state $m^{(t+1)}$.

Figure 2.1.: Reinforcement learning framework for molecular generation, illustrating the interaction between the agent, environment, and objective function

This interaction continues iteratively until a termination action is selected, yielding a complete molecule. The generated molecule is then evaluated using an *objective function*, which assigns a score reflecting how well it satisfies the desired properties.

The objective of the agent is to learn a policy that maximizes the expected value of this score over generated molecules.

In contrast to standard reinforcement learning settings, where rewards are typically assigned at each step of the interaction, in this framework the reward is only available after the full molecule has been generated. That is, the quality of a trajectory can only be evaluated once the generation process is terminated.

As a result, the learning process relies on associating sequences of actions with the final outcome. In particular, action sequences that lead to high-scoring molecules are reinforced, encouraging the agent to reproduce similar trajectories in the future.

2.2. Molecular Graph

In this workflow, a molecule (**Figure 2.2.a**) is represented as a hydrogen-suppressed graph (**Figure 2.2.b**), where nodes correspond to atoms and edges correspond to chemical bonds. To

define such a graph, three components need to be specified. First, the set of atom types that can be used to construct the molecule must be defined – *the atom alphabet* (**Figure 2.2.c**). Second, the set of atoms specifying the composition of an individual molecule – *the atom sequence* (**Figure 2.2.d**). Third, a *bond matrix* (**Figure 2.2.e**) is used to explicitly encode the connectivity between atoms and the corresponding bond types in this molecule.

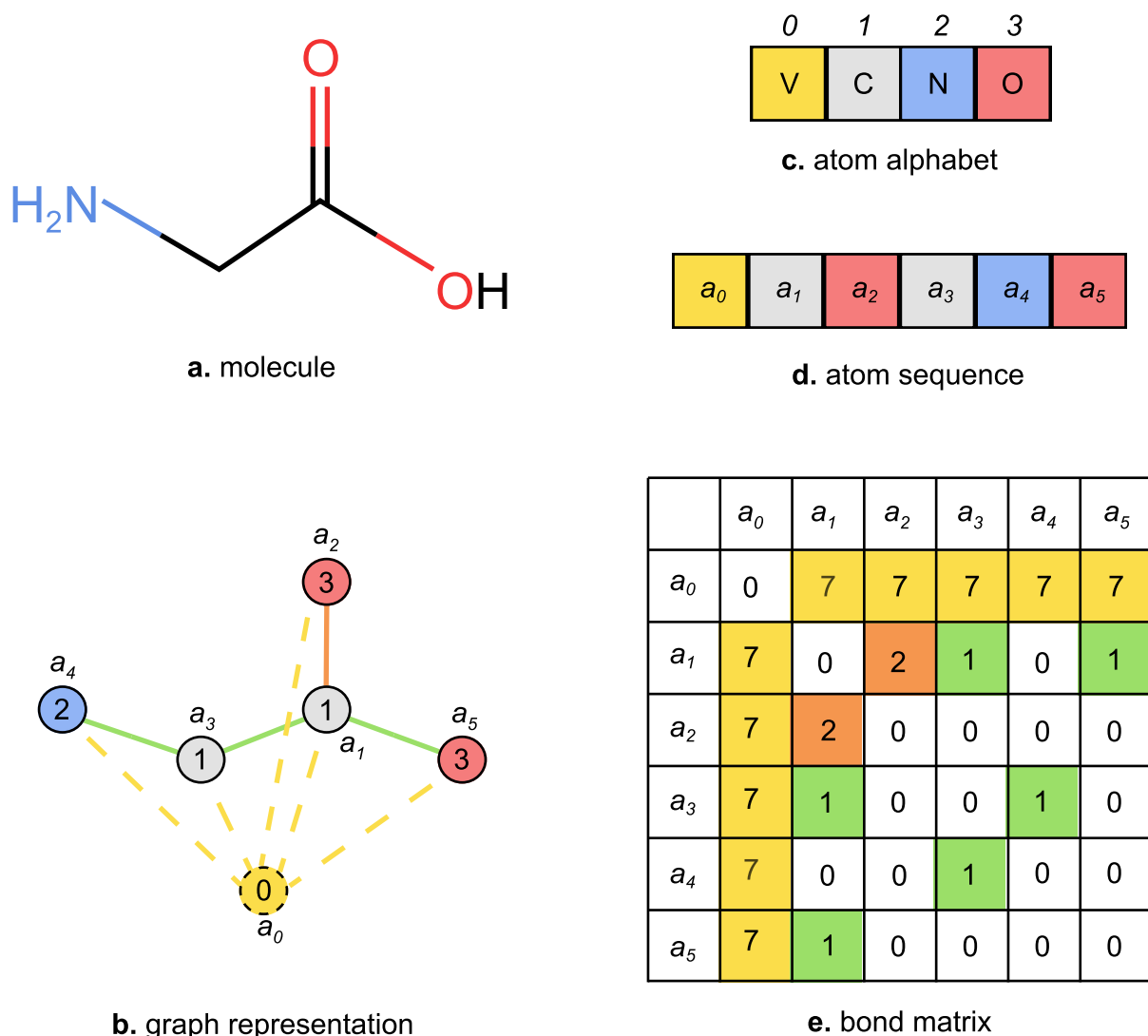


Figure 2.2.: Representation of a molecule as a hydrogen-suppressed graph

Let $\Sigma = (\Sigma_1, \dots, \Sigma_k)$ denote the set of possible atom types. In this setting, a molecule consisting of n atoms can be represented by a pair $(\mathbf{a}, \mathbf{B})$, where $\mathbf{a} = (a_1, \dots, a_n)$ is a vector indicating the atom types, with a_i corresponding to the i -th atom of type Σ_{a_i} , and $\mathbf{B}$ is a bond matrix describing the connections between atoms.

In particular, $\mathbf{B} \in \mathbb{N}_0^{n \times n}$ is a symmetric categorical matrix with a zero diagonal. Here, 0 denotes the absence of a bond, values $1, \dots, 6$ correspond to different bond types, 7 denotes a virtual bond connecting atoms to the virtual node, and 8 is reserved for padding in batched tensor

representations. The zero diagonal reflects the fact that an atom cannot be connected to itself. For real atoms, each atom is required to be connected to at least one other real atom in a chemically valid molecule.

An important property of molecular graph representations is their invariance to permutations of node indices. Since the ordering of atoms in a graph is arbitrary, any valid representation or model operating on molecular graphs should produce identical outputs regardless of how the nodes are indexed.

In addition, molecular representations should be invariant to geometric transformations such as rotations and translations in three-dimensional space. Since the chemical identity of a molecule does not depend on its spatial orientation, the model should produce identical predictions for any rotated or translated version of the same molecule.

These invariance properties are naturally satisfied by representing molecules as graphs without explicit spatial coordinates and by employing architectures that are permutation-invariant with respect to node ordering.

Not all graphs defined in the above manner correspond to chemically valid molecules. To ensure chemical validity, additional constraints must be imposed. In particular, atoms are required to satisfy valence constraints, such as the octet rule, which limits the number of bonds that each atom can form. In this work, we restrict our attention to molecular graphs that satisfy these constraints.

This framework can be naturally extended to incorporate additional chemical features. For example, ionization can be modeled by introducing atom types with modified bonding capacities, such as C^+ and C^- . Similarly, properties such as chirality can be incorporated by extending the atom representation.

In practice, the chemical properties of atoms are defined through a predefined configuration. Each atom type is associated with attributes such as atomic number, valence, formal charge, and optional properties including chirality. These attributes determine the allowed bonding behavior and enable enforcement of valence constraints during molecule construction. This design also allows for flexible extension of the atom set without modifying the underlying model.

An additional distinctive feature of this representation is the introduction of a virtual atom (**Figure 2.2.b**), node a_0 , which is connected to all real atoms in the graph via virtual bonds (**Figure 2.2.e**, denoted as “7”). This virtual node serves as a mechanism for aggregating graph-level information.

In particular, the virtual atom collects information from all nodes, enabling the model to construct a global representation of the molecule. This representation is then used to perform graph-level predictions.

This design is particularly important in the context of molecular generation, where decisions such as adding a new atom or terminating the generation process are formulated as classification tasks at the graph level.

2.3. Sequential Molecular Graph Design

Based on the graph representation introduced above, molecular generation can be formulated as a sequential graph construction process. Starting from an initial molecular graph, which must contain at least one node, the model iteratively applies actions that modify the current molecular graph. These actions may (a) add a new atom, (b) create a bond between existing atoms, or (c) leave the graph unchanged, thereby terminating the generation process.

More formally, the generation procedure can be formulated as a sequential Markov decision process (MDP) [30, 7]. A key property of a Markov process is that the next state depends only on the current state and the chosen action, and not on the sequence of previous states. In the context of molecular generation, this means that each decision is made solely based on the current molecular graph, without requiring access to the full history of how the molecule was constructed.

In this framework, the current molecular graph represents the state, and each action modifies this state. At each step t , the model observes the current molecular graph $\mathbf{m}^{(t)}$ and selects an action $\mathbf{x}^{(t)}$. Applying this action produces a new molecular graph $\mathbf{m}^{(t+1)}$. Repeating this procedure yields a sequence of intermediate molecular graphs until a special termination action is selected.

As discussed above, three types of actions on the molecular graph are available:

1. **Atom addition.** The model may add a new atom of a selected type and connect it to an existing atom in the graph with a bond of a specified order. Formally, this action is defined as

$$\mathbf{x}^{(t)} = \text{ADDATOM}(j, l, o),$$

where j denotes the index of an atom type from the alphabet, l denotes an existing node in the molecular graph, and o denotes the bond order. This action results in appending a new atom to the atom sequence and extending the bond matrix to include the newly created bond.

2. **Bond addition.** The model may introduce a new bond of a specified order between two existing atoms that are not yet connected, potentially forming a cycle. Formally, this action is defined as

$$\mathbf{x}^{(t)} = \text{ADDBOND}(j, l, o),$$

where j and l denote two unconnected atoms in the graph and o is the bond order. This action updates the bond matrix to reflect the newly added bond, while the atom sequence remains unchanged.

3. **Termination.** The model may select a special action, formally denoted as

$$\mathbf{x}^{(t)} = \text{DONTCHANGE},$$

which leaves the current molecular graph unchanged ($\mathbf{m}^{(t+1)} = \mathbf{m}^{(t)}$) and marks the generation process as complete.

These actions define the action space of the model. In addition, the model ensures that all actions applied to the molecular graph do not violate valence constraints, thereby guaranteeing chemical validity at each step of the generation process.

Furthermore, hydrogen atoms are treated implicitly. Consequently, adding a new atom or bond can be interpreted as replacing one or more implicit hydrogens in the current structure. This allows the model to operate directly on the heavy-atom graph while preserving chemical consistency. In other words, although the space of all possible atom combinations is large, the model operates within the subspace of chemically valid molecules and does not leave it during generation.

In practice, this is achieved through action masking, where invalid actions are removed from the model’s output distribution. This corresponds to assigning zero probability to such actions in the distribution over all possible actions at a given step.

Alternatively, one could assign a value of negative infinity to the objective function for molecules that violate chemical constraints after the generation process. However, the former approach significantly reduces the search space, as the model does not need to explore invalid trajectories during generation, thereby improving computational efficiency.

2.4. Molecular Optimization

The goal of molecular generation is to construct molecules that maximize a predefined objective function. Formally, this problem can be defined as

$$m^* \in \arg \max_{m \in \mathcal{M}} f(m),$$

where $\mathcal{M}$ denotes the space of valid molecules and $f : \mathcal{M} \rightarrow \mathbb{R} \cup \{-\infty\}$ is an objective function that assigns a score to each molecule, mapping chemically invalid molecules to $-\infty$.

This problem can be formulated within a reinforcement learning framework, where a policy π_θ maps a molecule $m \in \mathcal{M}$ to a probability distribution over possible actions. The objective is then

to learn a policy π_θ that maximizes the expected value of the objective function over generated molecules:

$$\mathbb{E}_{x^{(0)}, \dots, x^{(T)} \sim \pi_\theta} \left[f \left(\left(m^{(0)}, x^{(0)}, \dots, x^{(T)} \right) \right) \right],$$

where $m^{(0)}$ denotes the initial molecule, $(x^{(0)}, \dots, x^{(T)})$ represents the sequence of actions constructing the final molecule $m^{(T)}$, and $x^{(T)}$ corresponds to the DONTCHANGE action terminating the generation process.

In this work, the objective function f is defined using the goal-directed benchmarks from the GuacaMol framework [20]. In this setting, each molecule is assigned a score that reflects how well it satisfies a predefined set of desired properties.

The GuacaMol framework comprises twenty benchmark tasks of five different types. In *similarity tasks*, the goal is to generate molecules similar to a given target compound. In *rediscovery tasks*, the objective is to reproduce a specific target molecule. *Isomer tasks* require generating molecules that match a given molecular formula, while *median molecule tasks* aim to find molecules that are simultaneously similar to multiple reference structures.

A particularly important class of tasks within the GuacaMol benchmark are *multi-property optimization (MPO) tasks*. In these tasks, the objective function is defined as a combination of several criteria that must be satisfied simultaneously. Typically, MPO tasks include a mixture of similarity constraints, physicochemical properties such as $\log P$ and TPSA, and structural requirements. Each component is transformed into a normalized score, and the final objective is obtained by combining these scores, often using a geometric mean.

Such tasks are more challenging than single-objective optimization, as they require the model to balance potentially competing objectives. As a result, MPO benchmarks provide a more realistic evaluation of the model’s ability to perform molecular design in practical scenarios.

In general, GuacaMol provides a set of benchmark tasks designed to be sufficiently challenging to assess the fidelity of generated molecules and to enable a meaningful comparison between complex deep learning models.

2.5. Learning Algorithm

The exploration of chemical space is a highly challenging problem due to its vast combinatorial complexity. Efficient exploration is crucial for identifying high-quality molecules, as naive or poorly guided search strategies are unlikely to discover promising regions of the chemical space.

Classical graph search methods, such as Monte Carlo Tree Search (MCTS) and beam search, exhibit several limitations in this context. MCTS relies on repeated simulations and value back-propagation, which makes it computationally expensive and difficult to scale to high-dimensional spaces such as molecular graphs [7].

Beam search, in contrast, is computationally efficient but deterministic, retaining only the top- k most probable sequences at each step. As a result, it suffers from limited diversity and often converges prematurely to suboptimal regions of the search space [31].

These limitations are particularly problematic in molecular generation, where high-scoring molecules are often difficult to discover when the search is restricted to a limited number of locally optimal decisions. Therefore, effective search strategies must balance exploration and exploitation while remaining computationally efficient.

To overcome these limitations, the present work employs the TASAR algorithm [32], which can be viewed as an extension of stochastic beam search [33]. In contrast to standard beam search [34], stochastic beam search is not deterministic. By perturbing the search process with Gumbel noise, it introduces additional diversity into the sampled trajectories and thereby improves exploration of the search space.

Building on this idea, TASAR incorporates the “take a step and reconsider” strategy. Once a promising trajectory has been identified, the root of the search tree is shifted forward along this trajectory by a predefined step size, while the remaining part of the tree is discarded. The newly reached node is then treated as the new root, and the search process is restarted from this point.

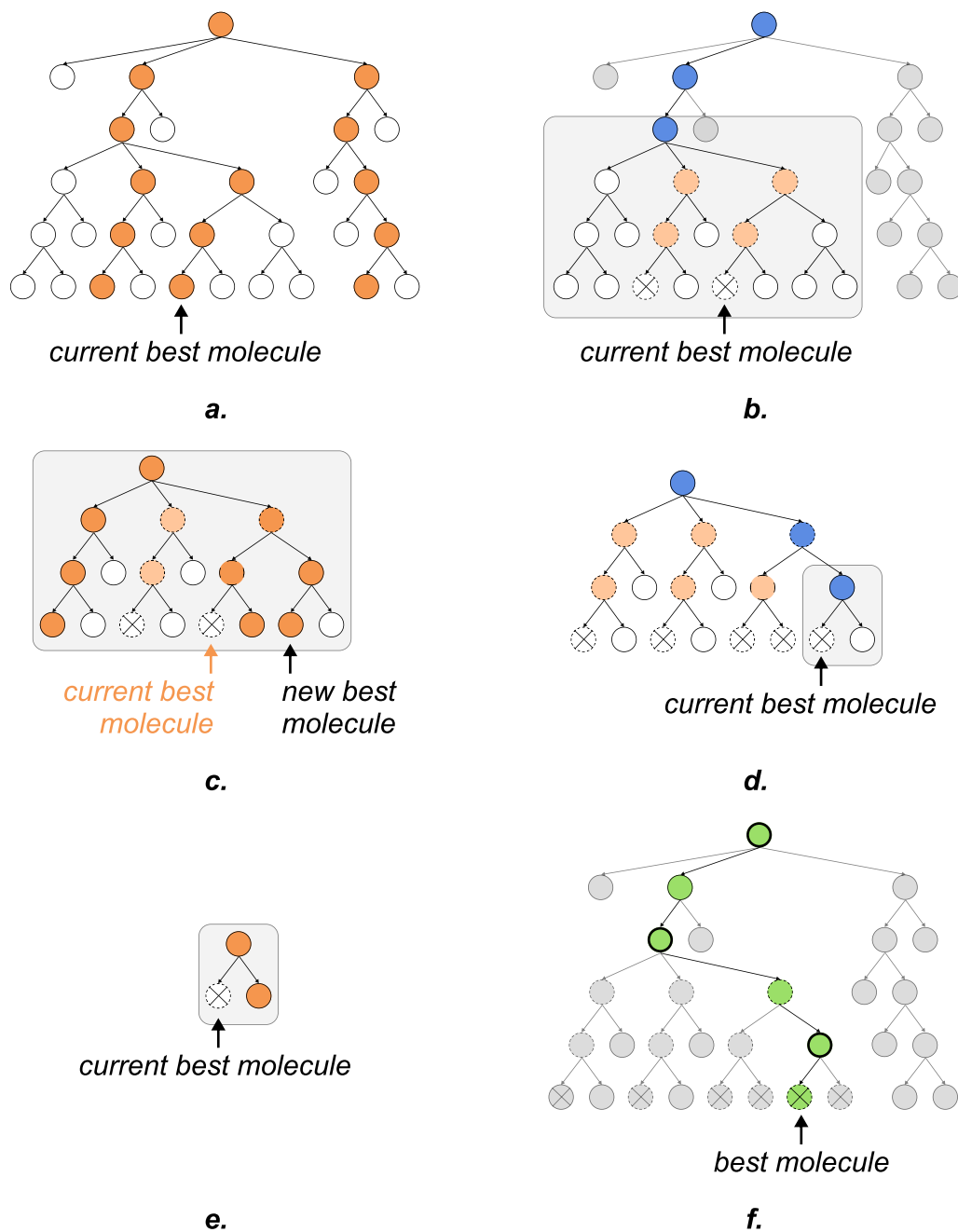
In this way, the algorithm repeatedly commits to the currently most promising partial solution while still allowing further exploration of alternative continuations.

An important component of this procedure is sampling without replacement [35, 36]. Once a leaf has been sampled, its probability mass is set to zero. In addition, the probability mass of that sampled leaf is subtracted from all of its ancestor nodes. As a result, already explored trajectories cannot be sampled again, which forces the algorithm to consider previously unseen alternatives.

Overall, this yields a conceptually simple and intuitive search procedure controlled by only two main hyperparameters: the beam width and the step size. At the same time, it enables efficient exploration of the solution space by combining focused search along promising trajectories with diverse sampling without replacement.

To illustrate the behavior of the algorithm, we consider the schematic representation of the chemical space shown in **Figure 2.3**. In this example, the algorithm operates with a beam width $\beta = 3$ and a step size $\sigma = 2$.

At the initial stage (**Figure 2.3.a**), three trajectories are sampled without replacement from the current policy. Among these, the trajectory yielding the highest-scoring molecule is identified as the current best trajectory.

**COLOR CODING:**

- – sampled trajectories
- – current best trajectory followed for s steps
- – nodes with reduced probability mass (already explored candidates)
- ⊗ – nodes with zero probability mass
- – discarded nodes (pruned from the search tree)
- – nodes belonging to the best-found trajectory
- – nodes selected as new roots during the search

Figure 2.3.: Example of sequence decoding using the TASAR algorithm with beam width $\beta = 3$ and step size $\sigma = 2$

Next, the algorithm follows the best trajectory for two steps (**Figure 2.3.b**), as defined by the step size σ , thereby setting a new root of the search tree. At this point, two possible cases arise for the remaining sampled trajectories.

In the first case, a trajectory does not share any nodes with the selected best trajectory. Such trajectories lie entirely in the part of the search tree that is discarded after the root shift.

In the second case, a trajectory partially overlaps with the selected best trajectory. In this situation, the probability mass of the sampled leaf is removed from its ancestor nodes in the normalized probability distribution, and the probability of the sampled leaf itself is set to zero. The same procedure is applied to the best trajectory, ensuring that already explored paths are not revisited in subsequent iterations.

In the next iteration (**Figure 2.3.c**), three new trajectories are sampled from the updated root of the search tree. As before, sampling is performed without replacement. Among these trajectories, a new best molecule is identified, which outperforms the previously selected one.

Consequently, the corresponding trajectory is marked as the new best trajectory, and the procedure described above is repeated (**Figure 2.3.d**). In particular, the root of the search tree is shifted along this trajectory, the probability mass of sampled leaves is set to zero, and the corresponding probability mass is subtracted from their ancestor nodes.

In the final stage, only a small subregion of the chemical space remains to be explored (**Figure 2.3.e**). At this point, only a single trajectory can be sampled, and it does not yield a molecule that improves upon the current best solution. Consequently, the search process terminates, and the best trajectory identified during the procedure is returned (**Figure 2.3.f**).

While high-quality molecules are discovered during the search process, it is essential to leverage them effectively for training the policy network. In this workflow, a learning framework that combines key ideas from the cross-entropy method (CEM) [37] and self-imitation learning (SIL) [38] is used.

From the cross-entropy method, it inherits the sampling-based optimization strategy, where trajectories are generated from the current policy, and only the top-performing ones are selected. The policy is then updated to increase the likelihood of these high-quality trajectories, encouraging the model to reproduce successful decisions.

From self-imitation learning, the idea of reusing past successful trajectories is used. By reinforcing previously discovered high-scoring solutions, the model is able to retain useful behaviors across iterations and improve training stability.

The combination of these approaches enables the model to both exploit newly discovered high-quality molecules and preserve previously learned knowledge. As a result, the policy is progressively guided toward more promising regions of the chemical space, leading to more efficient and effective exploration.

Summarizing the above, the overall learning procedure can be described as follows:

1. We begin with a policy network parameterized by θ , which can be initialized either randomly or using pretraining on existing molecular data. The generation process starts from an initial molecule, typically consisting of a single carbon atom.
2. During training, we maintain a set of the best molecules discovered so far, denoted as BESTFOUND. This set is initialized with the starting molecule and is updated throughout the training process.
3. At each training epoch, action sequences are sampled from the current policy using the TASAR procedure. These trajectories are evaluated using the objective function, and the most promising one is selected and followed for a predefined number of steps. From the resulting partial trajectory, alternative continuations are explored, allowing further refinement of the solution. This procedure promotes both diversity and focused exploration of the search space.
4. The set BESTFOUND is then used as a training dataset for supervised learning. In particular, the policy network is trained to predict the next action given a partial molecular graph, using a cross-entropy loss. This is analogous to training autoregressive models, where the model learns to reproduce high-quality action sequences.
5. The updated policy is then used in the next epoch, and the procedure is repeated iteratively. Over time, this process guides the model toward generating increasingly high-quality molecules.

2.6. Policy Network Architecture

The policy network takes a molecular graph as input and outputs a probability distribution over the possible next actions. In this work, we consider two different architectures for this purpose.

As a baseline, we employ a transformer-based model inspired by the Graphormer architecture [39], which has been shown to be effective for graph-structured data. In addition, we propose an alternative architecture based on xLSTM, which is specifically adapted to the molecular graph generation setting.

Both models operate on the same molecular representation and share the same input and output interface, allowing for a direct comparison within the proposed framework. In both cases, the learned latent representations are used to parameterize a probability distribution over the action space, enabling the model to select the next modification of the molecular graph.

2.6.1. Splitting Actions

To enable more precise and structured decision-making during molecule generation, each modification action is decomposed into a sequence of three sub-actions. Each of these sub-actions is predicted via a separate forward pass of the policy network.

- At action level 0, the agent decides whether to terminate the generation process or to modify the current molecular graph. This corresponds to choosing between the DONTCHANGE action and a modification action. In the latter case, the agent determines the first parameter j . Depending on the selected operation, this parameter has different interpretations: for ADDBOND, j corresponds to an atom selected from the existing molecular graph, whereas for ADDATOM, j denotes an atom type selected from the predefined atom alphabet. After selecting j , the molecular graph is updated accordingly, and the updated state is passed again through the policy network.
- At action level 1, the agent selects a second atom l from the current molecular graph, indicating the formation of a bond between atoms j and l . This information is incorporated into the graph representation, followed by another forward pass through the network.
- Finally, at action level 2, the agent determines the bond order o between atoms j and l , completing the definition of the action.

The entire process can thus be viewed as a sequential decision pipeline, where each step conditions on the previously selected components (**Figure 2.4**).

2.6.2. Node Embedding and Feature Augmentation

As discussed above, a molecule is represented as an unordered molecular graph defined by an atom sequence and a bond matrix. However, this structural information alone is not sufficient for the policy network to make informed decisions during the graph construction process.

In addition to the molecular structure, the policy network must incorporate contextual information related to the current stage of the generation procedure. In particular, as introduced in the previous section, the model needs to account for the current action level, as well as the intermediate decisions made at previous stages, such as the atom selected at action level 0 and the atom selected at action level 1.

Furthermore, the network must consider structural properties of the current graph, such as the degree of each atom, which constrains the set of valid actions. Incorporating this additional information enables the policy network to produce context-aware decisions and ensures consistency with chemical constraints during molecule generation.

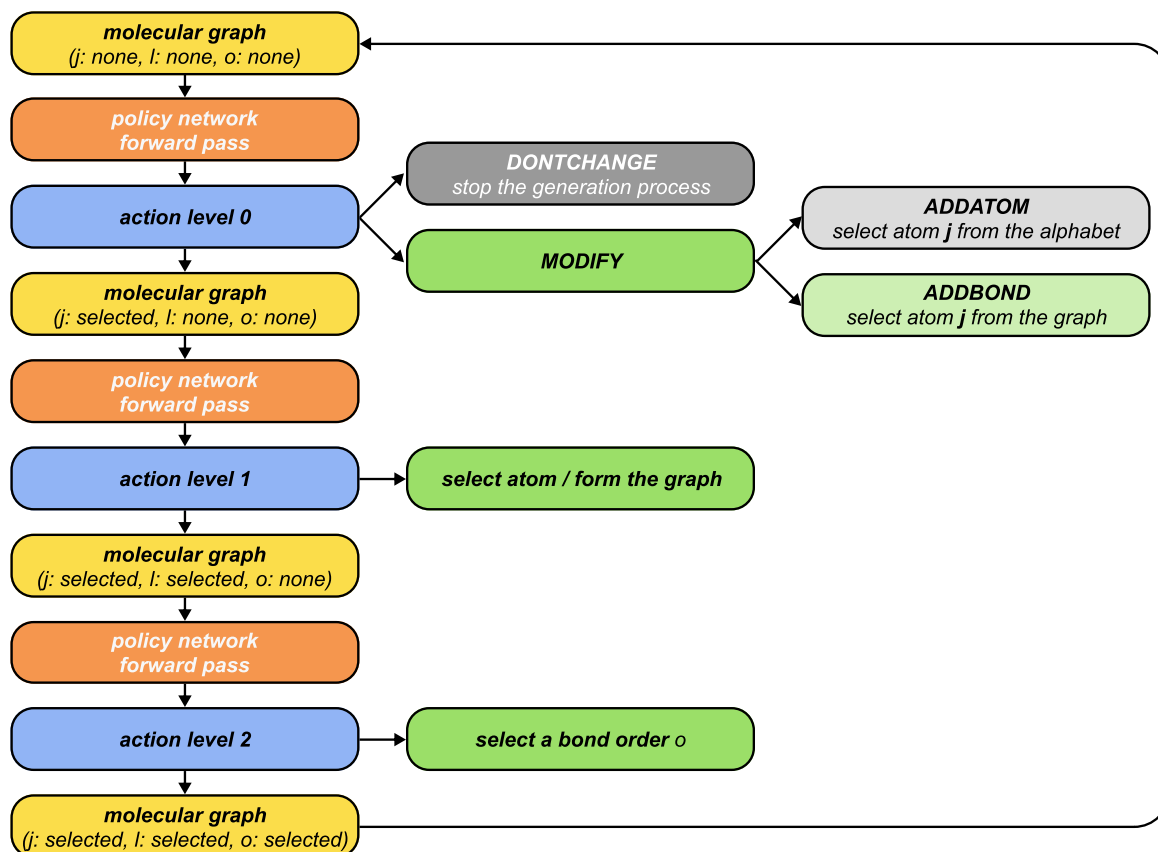


Figure 2.4.: Decomposition of a molecule generation action into a sequence of sub-actions

To incorporate this information into the model, each feature is first mapped into a vector space using a dedicated embedding layer. This includes the atom sequence, the current action level, the indicators of previously selected atoms at action levels 0 and 1, as well as the degree of each atom. The treatment of bond information depends on the specific model architecture and is described in the corresponding sections below.

The resulting embeddings are then combined through element-wise summation to form the final node representations. Importantly, different components are applied selectively depending on the node type. The embedding corresponding to the current action level is added only to the virtual node, allowing it to capture global information about the generation stage. In contrast, the embeddings encoding previously selected atoms (at action levels 0 and 1) and atom degrees are added to the representations of real atoms in the molecular graph.

This design enables the model to integrate both structural and decision-related information into a unified representation, which is subsequently used by the policy network for action prediction. The entire process is illustrated in **Figure 2.5** and can be formally expressed by the following equation:

$$\left(\hat{a}_0 + r, \hat{a}_1 + z_1 + w_1^{(0)} + w_1^{(1)}, \dots, \hat{a}_n + z_n + w_n^{(0)} + w_n^{(1)} \right),$$

where $a_0, a_1, \dots, a_n$ denote the atom sequence, with a_0 corresponding to the virtual node and $a_i, i = 1, \dots, n$, corresponding to the atoms of the molecular graph. Each atom is mapped to a latent representation $\hat{a}_i \in \mathbb{R}^d$ using an embedding layer.

Furthermore, $r \in \mathbb{R}^d$ denotes the embedding corresponding to the current action level. For each atom $i = 1, \dots, n$, $z_i \in \mathbb{R}^d$ represents the embedding of the atom degree, and $w_i^{(0)}, w_i^{(1)} \in \mathbb{R}^d$ denote embeddings indicating whether the i -th atom was selected at action levels 0 and 1, respectively.

Figure 2.5.: Node embedding and feature augmentation

2.6.3. Transformer-based Node Representation Model

As a baseline architecture, we employ a transformer-based model (**Figure 2.6**) inspired by the Graphormer architecture, which has been shown to be effective for graph-structured data.

The model operates on the augmented node representations $(e_0, e_1, \dots, e_n) \in \mathbb{R}^{(n+1) \times d}$, where e_0 corresponds to the virtual node.

The sequence of node representations is processed by a stack of transformer layers with multi-head self-attention and position-wise feed-forward networks. To ensure permutation equivariance with respect to the ordering of atoms, no positional encodings are used. Instead, the model relies entirely on structural information encoded in the input features and the bonding structure.

A key advantage of the transformer-based architecture is its ability to incorporate pairwise relational information directly into the attention mechanism. In particular, the bond matrix $B \in \mathbb{R}^{(n+1) \times (n+1)}$, encoding bond types between atoms, is introduced as a learnable bias in the attention computation.

Let $A = (A_{ij})$ denote the attention score matrix for a given attention head. Then, the attention scores are modified as:

$$A_{ij} \leftarrow A_{ij} + b_{ij},$$

where $b_{ij} \in \mathbb{R}$ is a learnable scalar that depends only on the bond type between nodes i and j .

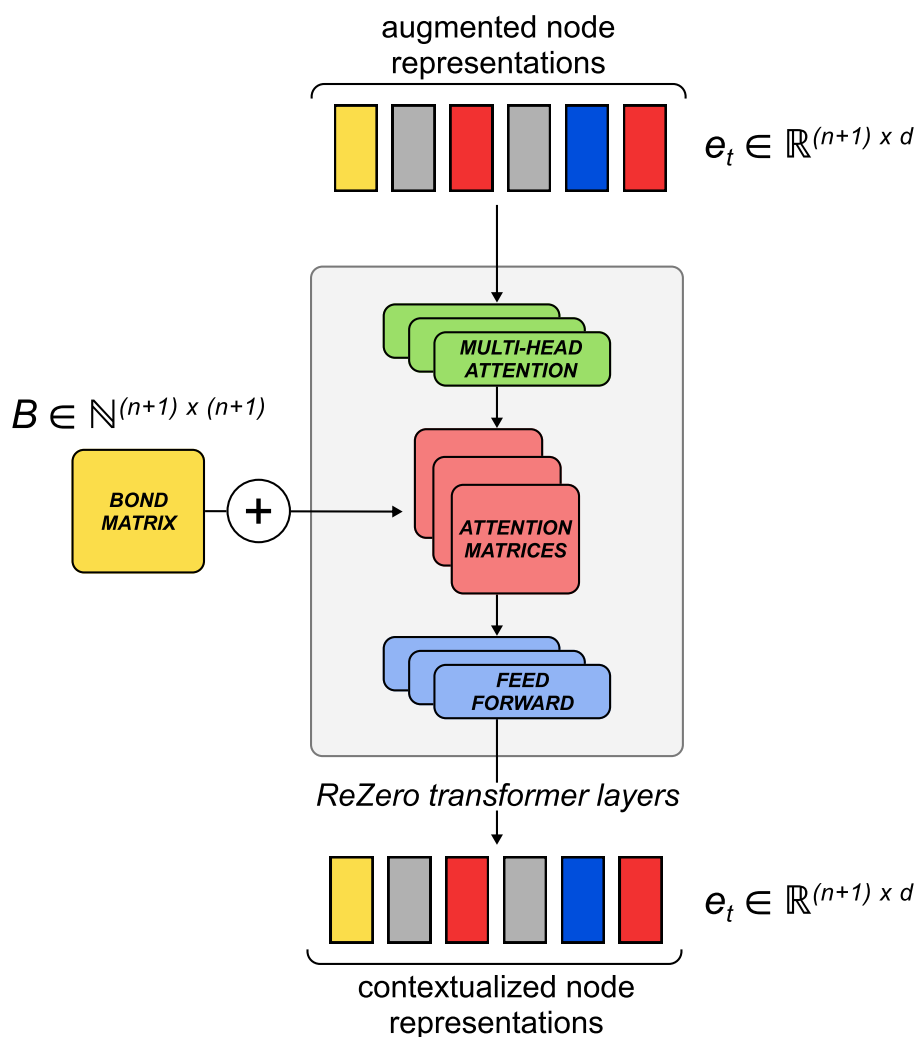


Figure 2.6.: Transformer-based node representation model (single transformer block)

This allows the model to explicitly account for chemical bonding information when computing attention weights. In particular, different bond orders correspond to different learnable bias values, and these parameters are learned independently across attention heads and layers.

The resulting attention scores are then normalized using the softmax operation and used to compute the weighted aggregation of node representations. The transformer layers are implemented using ReZero normalization [40], which improves training stability and enables deeper architectures.

The main architectural hyperparameters of the model are the number of transformer blocks and the number of attention heads. These control the depth and the expressiveness of the model, respectively.

2.6.4. xLSTM-based Node Representation Model

Despite the strong performance of transformer-based models on molecular graph tasks, they suffer from a fundamental limitation due to their quadratic computational complexity with respect to the number of nodes in the molecular graph. This becomes particularly restrictive in scenarios where longer action sequences or larger molecular structures need to be considered.

To address this limitation, we propose to replace the transformer-based representation module with a hybrid architecture built around an xLSTM-based sequence model (**Figure 2.7**). The xLSTM architecture can be viewed as a generalization of the classical LSTM, retaining its key property of maintaining a recurrent memory through a cell state, while incorporating ideas from linear attention mechanisms [41].

Theoretically, xLSTM enables efficient modeling of long-range dependencies while achieving improved computational scalability compared to standard self-attention. This makes it a suitable candidate for sequential decision-making problems such as molecular graph generation, where both long-range interactions and computational efficiency are important.

In contrast to transformer-based models, the xLSTM architecture does not provide a direct mechanism for incorporating the bond matrix into its computation. While transformer-based models can naturally integrate such information as a learnable bias in the attention scores, the sequential nature of xLSTM requires a different approach.

To account for bonding information, we incorporate structural graph information directly into the node representations prior to sequence modeling. In particular, the bond matrix is processed through a stack of graph neural network (GNN) layers, which perform message passing [42] over the molecular graph and enrich node representations with information about their local neighbor atoms and bonds, representations from different GNN layers are aggregated using a Jumping Knowledge mechanism [43].

The resulting representations are subsequently projected to the model dimension via a linear projection layer and processed by an xLSTM block stack.

The main architectural hyperparameters of the model include the number of GNN layers, the number of xLSTM blocks, and the number of xLSTM heads. Additionally, dropout can be applied to the GNN component of the network.

Graph Neural Network Encoder

To incorporate structural information from the molecular graph into node representations, we employ a graph neural network (GNN) encoder based on a message passing mechanism. The input graph is represented by a node sequence and a bond matrix, which defines the connectivity and bond types between atoms.

Figure 2.7.: xLSTM-based node representation model

Instead of performing dense computations over all pairs of nodes, we adopt a sparse message passing formulation, where messages are exchanged only between nodes connected by chemical bonds [42]. The encoder operates on the resulting graph structure

$$G = (V, E),$$

where nodes correspond to atoms and edges correspond to chemical bonds.

The architecture of a single GNN layer is illustrated in **Figure 2.8**.

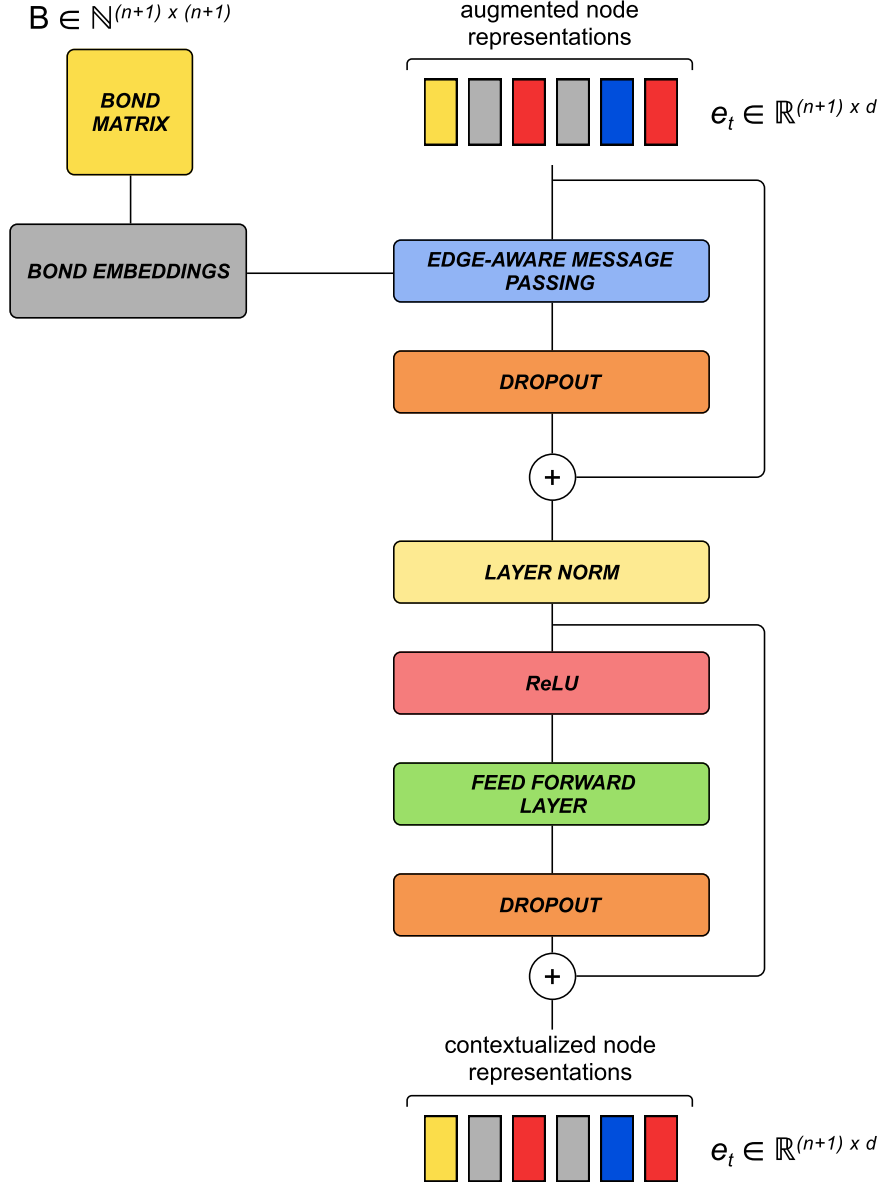


Figure 2.8.: Graph neural network encoder layer

Let $h_i^{(l)} \in \mathbb{R}^d$ denote the representation of node i at layer l , and let $e_{ij} \in \mathbb{R}^d$ denote the embedding of the bond between nodes i and j . The initial node representations $h_i^{(0)}$ are obtained from the augmented node embeddings described in the Node Embedding and Feature Augmentation section (**Figure 2.5**).

At each GNN layer, messages are computed from neighboring nodes and aggregated to update node representations. For each edge $(j, i) \in E$, a message is defined as:

$$m_{j \rightarrow i}^{(l)} = \phi \left(W_m \left(h_j^{(l)} + e_{ij} \right) \right),$$

where $W_m \in \mathbb{R}^{d \times d}$ is a learnable weight matrix and $\phi(\cdot)$ denotes a non-linear activation function (ReLU in our implementation).

The incoming messages from all neighbors are then aggregated as:

$$m_i^{(l)} = \sum_{j \in \mathcal{N}(i)} m_{j \rightarrow i}^{(l)},$$

where $\mathcal{N}(i)$ denotes the set of neighboring nodes of node i .

The node representations are updated using a residual connection followed by normalization:

$$\tilde{h}_i^{(l)} = \text{LayerNorm} \left(h_i^{(l)} + \text{Dropout} \left(m_i^{(l)} \right) \right).$$

A position-wise feed-forward transformation is then applied:

$$h_i^{(l+1)} = \text{LayerNorm} \left(\tilde{h}_i^{(l)} + \text{Dropout} \left(W_o \phi \left(\tilde{h}_i^{(l)} \right) \right) \right).$$

where $W_o \in \mathbb{R}^{d \times d}$ is a learnable projection matrix.

The message passing process can be intuitively understood as illustrated in **Figure 2.9**. In this figure, the molecular graph is visualized from the perspective of a single atom a_1 , highlighting how information is propagated across the graph.

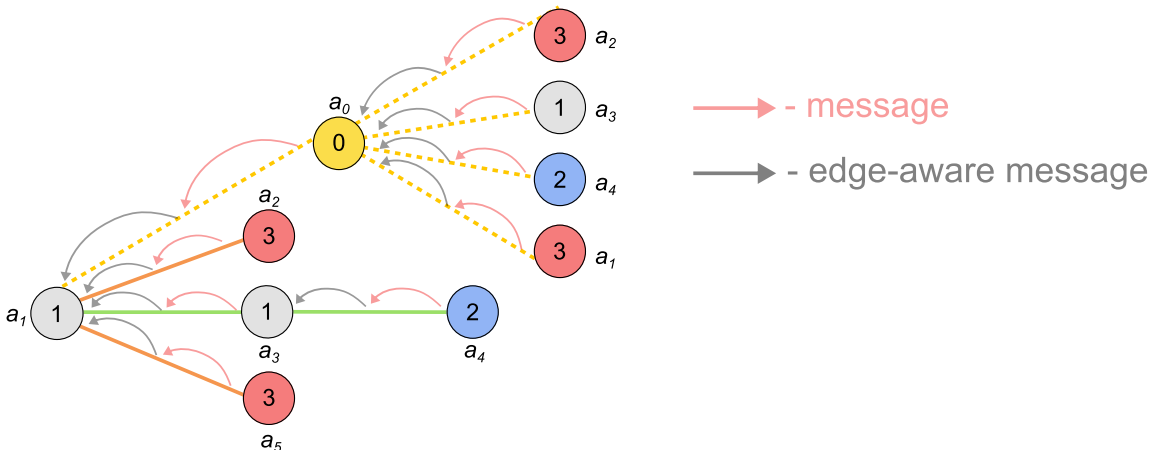


Figure 2.9.: Example of the message passing process in the proposed GNN

During a single message passing step, each node receives information from its immediate neighbors, i.e., nodes that are directly connected to it by chemical bonds. The messages are edge-aware, meaning that the information from neighboring nodes is augmented with the corresponding bond features, allowing the model to account for both node and edge attributes.

As a result, after one GNN layer, each node encodes information about its local neighborhood (i.e., its 1-hop neighbors). Stacking multiple GNN layers increases the receptive field: after k layers, each node incorporates information from nodes that are up to k hops away in the graph.

An important feature of the proposed architecture is the inclusion of a virtual node that is connected to all other nodes in the graph. During message passing, this virtual node aggregates information from all atoms in a single step. Consequently, already after the second layer, information from any node can be propagated to all other nodes via the virtual node, enabling efficient global information exchange across the molecular graph.

To capture information from multiple receptive field sizes, we employ a Jumping Knowledge mechanism that aggregates representations from all GNN layers:

$$h_i^{\text{JK}} = \text{CONCAT} \left(h_i^{(1)}, h_i^{(2)}, \dots, h_i^{(L)} \right),$$

where L is the number of GNN layers.

Finally, the concatenated representation is projected back to the model dimension:

$$h_i^{\text{proj}} = W_{\text{proj}} h_i^{\text{JK}},$$

where $W_{\text{proj}} \in \mathbb{R}^{d \times (L \cdot d)}$.

The resulting representations are then used as input to the subsequent xLSTM-based sequence model.

xLSTM-based Sequence Model

The node representations produced by the GNN encoder are further processed by a stack of xLSTM blocks, which serve as a neural representation model for capturing long-range dependencies across the molecular graph.

The output of the GNN

$$H^{\text{GNN}} \in \mathbb{R}^{(n+1) \times d}$$

is treated as a sequence of node representations and passed through the xLSTM-based model:

$$H^{\text{xLSTM}} = \text{xLSTM} \left(H^{\text{GNN}} \right).$$

The xLSTM block (**Figure 2.10**) is designed to closely resemble the structure of a transformer layer while replacing the attention mechanism with a recurrent alternative. Each block includes a projection layer, head-wise normalization, and a residual pathway that mirrors the design principles of transformer architectures.

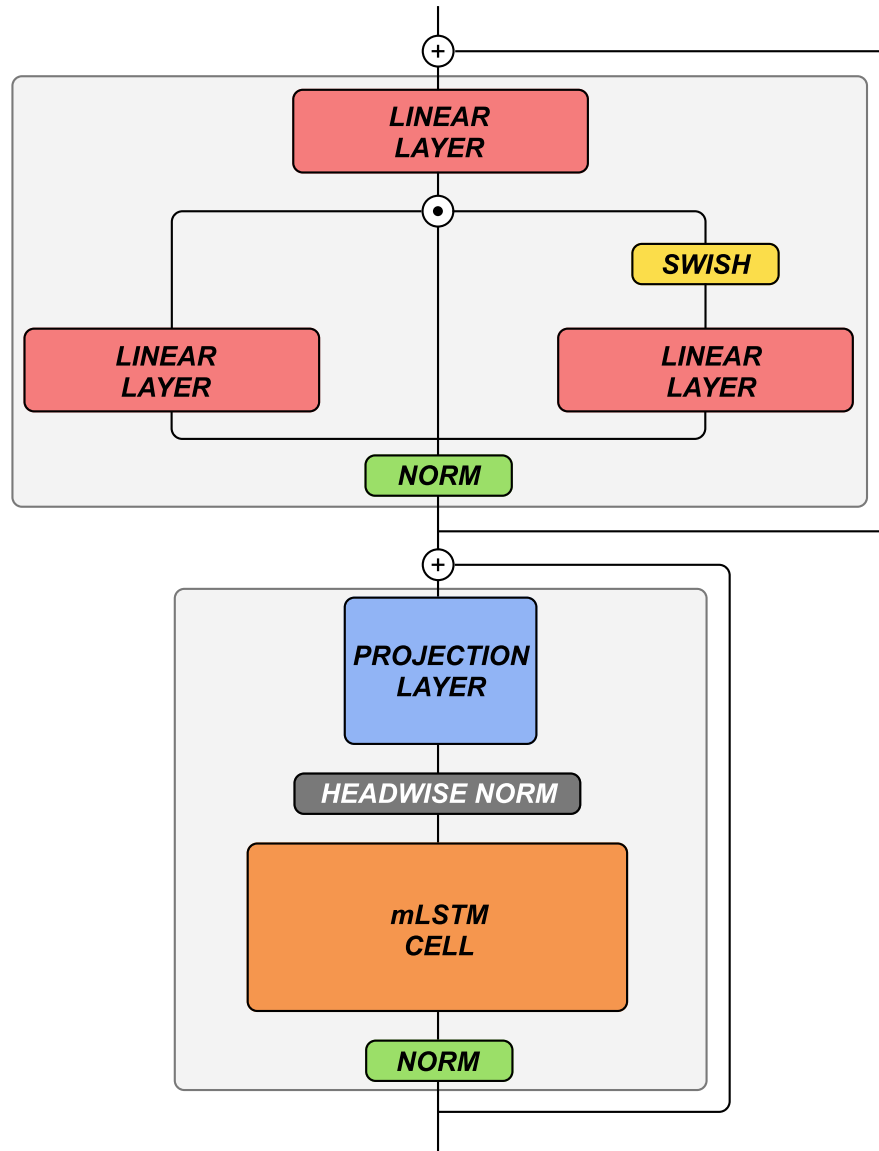


Figure 2.10.: Single xLSTM block

Multiple such xLSTM blocks are stacked to form the full sequence model, further reinforcing the analogy to transformer-based architectures. The core computational component of each block is the mLSTM cell, which is responsible for modeling dependencies across the sequence in a recurrent manner.

The mLSTM cell (**Figure 2.11**) extends the classical Long Short-Term Memory (LSTM) architecture by incorporating modifications inspired by linear attention, enabling more efficient modeling of long-range dependencies.

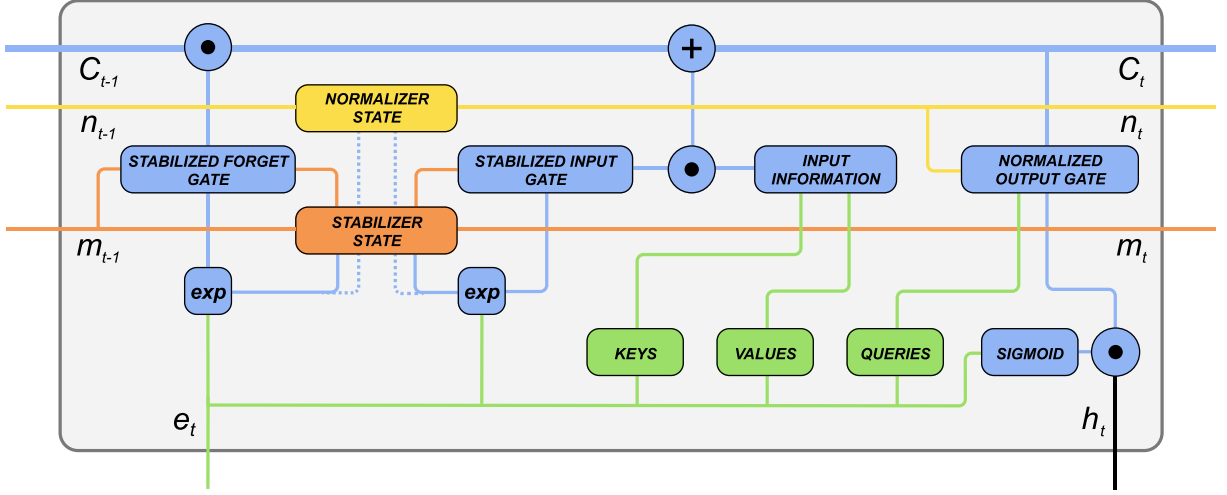


Figure 2.11.: Single mLSTM cell

The mLSTM stores information in a matrix $C_t \in \mathbb{R}^{d \times d}$, which enables the storage and retrieval of key-value associations.

At each step t , the memory is updated using a covariance-based rule:

$$C_t = f_t \odot C_{t-1} + i_t v_t k_t^\top,$$

where $k_t, v_t \in \mathbb{R}^d$ denote the key and value representations, respectively, and i_t, f_t are the input and forget gates.

Instead of relying solely on sigmoid and hyperbolic tangent activations, the mLSTM employs exponential gating mechanisms as:

$$i_t = \exp(\tilde{i}_t).$$

This modification allows the model to more effectively regulate the contribution of new and past information, enabling sharper updates to the memory state. In particular, exponential gating facilitates stronger suppression of irrelevant information and more selective incorporation of informative signals.

Furthermore, the mLSTM incorporates components analogous to queries, keys, and values from attention mechanisms as:

$$q_t = W_q e_t + b_q, \quad k_t = W_k e_t + b_k, \quad v_t = W_v e_t + b_v,$$

where e_t denotes the input embedding at sequence position t .

These representations enable the cell to perform content-based interactions over the sequence, effectively bridging recurrent architectures and linear attention models.

To ensure numerical stability under exponential transformations, additional internal states are introduced. Specifically, the mLSTM maintains a stabilizer state m_t (not to be confused with the molecular graph $m^{(t)}$) that prevents uncontrolled growth of gate values, defined as:

$$m_t = \max(\log(f_t) + m_{t-1}, \log(i_t)).$$

The stabilized input and forget gates are then computed as:

$$\begin{aligned}\tilde{i}_t &= \exp(\log(i_t) - m_t), \\ \tilde{f}_t &= \exp(\log(f_t) + m_{t-1} - m_t).\end{aligned}$$

This formulation prevents numerical overflow caused by exponential activations while preserving the relative scaling of the gating values.

In addition to the stabilizer state, the cell maintains a normalizer state that rescales intermediate representations before producing the final output:

$$n_t = f_t \odot n_{t-1} + i_t k_t.$$

Both stabilizer and normalizer are crucial for maintaining stable training dynamics.

To produce a cell output, the hidden state is computed as:

$$h_t = o_t \odot \tilde{h}_t, \quad \text{where} \quad \tilde{h}_t = \frac{C_t q_t}{\max(|n_t^\top q_t|, 1)},$$

and $q_t \in \mathbb{R}^d$ denotes the query representation.

An important distinction from the classical LSTM lies in the reformulation of the recurrent computation. In standard LSTM architectures, the hidden state h_t is propagated sequentially from one time step to the next, creating a strict dependency that limits parallelization during training.

In contrast, the mLSTM architecture reduces the reliance on such sequential hidden state propagation by restructuring the computation in a form that is more amenable to parallel execution. This enables significantly more efficient training, particularly on modern hardware accelerators. Moreover, the mLSTM design is specifically optimized for GPU execution, allowing the core operations to be efficiently implemented using CUDA kernels [44]. As a result, the model achieves improved computational efficiency.

2.6.5. Action-level Distributions

The policy network outputs a sequence of node representations

$$(e_0, e_1, \dots, e_n),$$

where $e_i \in \mathbb{R}^d$ corresponds to the i -th node of the molecular graph, and e_0 represents the virtual node.

These representations are used to parameterize three categorical probability distributions $P^{(0)}$, $P^{(1)}$, and $P^{(2)}$, corresponding to the three action levels (**Figure 2.12**). Graph-level decisions are predicted from the virtual node representation using projection functions denoted by g_r , whereas node-level decisions are computed independently for each node using functions denoted by ψ_r . The projection functions produce logits, which are masked to exclude invalid actions and then normalized with a softmax operation to obtain probability distributions over the corresponding action spaces.

Figure 2.12.: Action-level probability distribution

At action level 0, the model decides whether to terminate the generation process or to modify the molecule. The corresponding probability distribution over actions is defined as

$$P^{(0)} = (p_0^{(0)}, p_1^{(0)}, \dots, p_k^{(0)}, \tilde{\zeta}_1^{(0)}, \dots, \tilde{\zeta}_n^{(0)}) \in \mathbb{R}^{k+1+n},$$

where $p_0^{(0)}$ corresponds to the DONTCHANGE action, and $p_j^{(0)}$, $j = 1, \dots, k$, correspond to adding a new atom of type Σ_j (ADDATOM).

These probabilities are obtained by projecting the virtual node representation:

$$(p_0^{(0)}, \dots, p_k^{(0)}) = g_0(e_0), \quad g_0 : \mathbb{R}^d \rightarrow \mathbb{R}^{k+1}.$$

The remaining probabilities $\tilde{\zeta}_i^{(0)}$, corresponding to selecting an existing atom for bond formation (ADDBOND), are computed independently for each node:

$$\tilde{\zeta}_i^{(0)} = \psi_0(e_i), \quad \psi_0 : \mathbb{R}^d \rightarrow \mathbb{R}, \quad i = 1, \dots, n.$$

At action level 1, the model selects the second atom for bond formation. The corresponding probability distribution is defined as

$$P^{(1)} = \left(\xi_1^{(1)}, \dots, \xi_n^{(1)} \right),$$

where

$$\xi_i^{(1)} = \psi_1(e_i), \quad \psi_1 : \mathbb{R}^d \rightarrow \mathbb{R}.$$

At action level 2, the model determines the bond order between the selected atoms. The corresponding probability distribution is defined as

$$P^{(2)} = \left(p_1^{(2)}, \dots, p_y^{(2)} \right),$$

where y denotes the maximum bond order.

These probabilities are computed from the virtual node representation:

$$\left(p_1^{(2)}, \dots, p_y^{(2)} \right) = g_2(e_0), \quad g_2 : \mathbb{R}^d \rightarrow \mathbb{R}^y.$$

After an action is selected at a given level, the corresponding information is fed back into the model by updating the node representations via the embeddings (**Figure 2.5**). The updated state is then used for the subsequent forward pass when predicting the next action level.

Finally, invalid actions (e.g., those violating valence constraints) are masked by assigning them zero probability. This reduces the effective search space and improves training efficiency by preventing the model from exploring infeasible molecular structures.

2.7. Network Hyperparameters

For all case studies and experiments, we set the latent space dimension to $d = 512$ for both models.

The transformer-based model consists of 10 transformer layers with ReZero normalization, each featuring 16 attention heads. The xLSTM-based model also comprises 10 mLSTM layers with 16 mLSTM heads.

The transformer-based model contains 31,542,218 trainable parameters, while the xLSTM-based model contains 34,709,856 trainable parameters. Thus, the two architectures are comparable in terms of model capacity, enabling a fair comparison between attention-based and recurrent sequence modeling approaches.

2.8. Experimental Setup and Evaluation Protocol

The atom alphabet Σ was defined to include the most common elements occurring in organic molecules, namely C, N, O, F, P, S, Cl, Br, and I, along with variants accounting for ionization states and chirality.

Although the policy network is capable of learning from scratch, a pretraining stage was employed to initialize the model with prior knowledge of realistic molecular structures. For this purpose, we use molecules from the ChEMBL database represented as SMILES strings. The dataset is filtered to retain only molecules composed of atoms from Σ .

The initial dataset contains approximately 2.47 million molecules, which are split into training (~ 2.37 million molecules) and validation (0.1 million molecules) subsets. After filtering, the training set consists of 1,653,801 molecules, while the validation set contains 69,736 molecules.

Each SMILES string was converted into an action sequence under the used graph-based formulation used in this work. Since the mapping from SMILES to action sequences is not unique, a single valid sequence is selected arbitrarily for each molecule. These sequences are used to train the model in a self-supervised manner by predicting the next action given the previous ones.

Pretraining was performed using a dropout rate of 0.1. We consider two batch size configurations, 512 and 256, for both models. For a batch size of 512, each epoch consists of 3000 training batches and 750 validation batches. For a batch size of 256, the number of batches is doubled to 6000 for training and 1500 for validation.

The size of the resulting training dataset is determined by the conversion of molecules into action sequences. Starting from 1,653,801 molecules, the training set contains 155,976,882 datapoints. Similarly, the validation set, constructed from 69,736 molecules, comprises 6,564,912 datapoints.

To ensure consistent training across different batch size configurations, we control the effective number of datapoints processed per epoch. Pretraining was conducted for 1000 epochs for all models, during which each model processed approximately 1.5 billion datapoints, corresponding to roughly 10 effective passes over the training dataset.

The number of validation batches is set to 750 and 1500 for batch sizes of 512 and 256, respectively, ensuring that each validation cycle covers approximately 25% of the training volume.

Given a pretrained policy network, it can be used for molecule generation under different objective functions. In this setting, fine-tuning is employed to adapt the pretrained model to a specific task by incorporating feedback from a given objective function and guiding the generation process toward high-scoring molecules.

During fine-tuning, only the parameters of the final linear layers mapping node representations to the action space are updated, while the backbone of the model remains fixed. The initial molecule m_0 is set to a single carbon atom.

The fine-tuning process is governed by several groups of hyperparameters that control different aspects of the molecule generation procedure.

First, the maximum allowed molecule length defines the upper bound on the size of generated molecules and thus constrains the search space. By default, this parameter is set to 50 atoms.

Second, the parameter N_{best} determines the number of top-performing molecules retained in the BESTFOUND set, which serves as the source of training data during fine-tuning. In all experiments, this parameter is fixed to $N_{\text{best}} = 100$.

Third, the sampling procedure is controlled by parameters that regulate the degree of exploration during generation. In particular, the beam width β defines the number of candidate solutions considered at each step, while the step size σ determines how frequently alternative candidate trajectories are explored.

Finally, the degree of exploitation is governed by the number of training batches used for fine-tuning at the end of each epoch, which controls how strongly the model adapts to the currently best-performing molecules.

To establish a consistent reference point for subsequent analysis, we first evaluate all models across the full set of GuacaMol benchmark tasks under a baseline configuration with a beam width $\beta = 512$ and step size $\sigma = 12$, that is, after every four added atoms or bonds, TASAR explores alternative candidate solutions.

At the end of each epoch, the network is trained on 20 batches of size 64 datapoints, sampled uniformly from the BESTFOUND set.

This initial evaluation serves as a sanity check and provides a unified basis for comparing model behavior. Furthermore, the results obtained under this configuration are used to guide the selection of tasks for subsequent experiments, enabling a structured analysis across different levels of difficulty.

As the primary performance metric, we consider the highest-scoring molecule discovered during the optimization process. This metric reflects the maximum value of the objective function achieved by the model over the course of molecule generation. Following the GuacaMol benchmark, the score is normalized to lie in the range $[0, 1]$, where higher values indicate better performance.

To analyze the impact of fine-tuning under a fixed beam width, we fix the sampling parameters to $\beta = 512$ and $\sigma = 12$, which govern the degree of exploration, and vary the number of fine-tuning updates performed at the end of each epoch, which controls the degree of exploitation.

To assess the sensitivity of the fine-tuning procedure and to explore the balance between exploration and exploitation, we consider the standard configuration with 20 batches of size 64 datapoints, as well as configurations with 40 and 100 batches of the same size, and a setting corresponding to a single pass over the entire BESTFOUND set. These experiments are conducted for all considered model configurations.

To investigate the joint effect of fine-tuning and beam width tuning, we vary the number of fine-tuning batches per epoch following the procedure described above, while modifying the sampling parameters to encourage increased exploration: $\beta \in \{512, 1024, 2048\}$.

Furthermore, for these experiments, the reconsideration step size is reduced to $\sigma = 3$, allowing the model to revise intermediate decisions more frequently and explore alternative candidate trajectories.

In addition, the maximum allowed molecule length is increased from 50 to 70 atoms, providing additional flexibility for constructing more complex molecular structures.

Since this setup corresponds to a full factorial experimental design, the joint analysis is conducted on a subset of models. In particular, we consider one representative configuration for each architecture, selected based on the best performance observed under the baseline evaluation.

To evaluate the computational efficiency of both architectures, we measure the number of optimization epochs completed within a fixed wall-clock time budget of eight hours, providing insight into computational speed, training dynamics, and convergence behavior.

In addition, we analyze GPU memory consumption during pretraining and molecule generation.

To analyze the impact of fine-tuning intensity on computational efficiency, we use the same configurations as described above, considering configurations with 20, 40, and 100 batches, as well as a setting corresponding to a full pass over the BESTFOUND set.

To assess the impact of exploration on computational efficiency, we additionally vary the beam width, considering values $\beta \in \{512, 1024, 2048, 4096\}$, which represent moderate and extreme exploration regimes, respectively.

All experiments, including computational efficiency measurements reported in this work, are dependent on the specific software and hardware environment used for experimentation. In particular, the results were obtained using PyTorch 2.5.1 with CUDA 12.1 and cuDNN 9 within a containerized environment based on the official PyTorch CUDA 12.1 development image.

It is important to note that both training and inference efficiency can be significantly affected by low-level implementation details, including CUDA kernel optimizations, memory management strategies, and backend-specific performance improvements. These factors may differently impact transformer-based and recurrent-style architectures.

Therefore, the observed differences in performance and computational efficiency between transformer-based and xLSTM-based models should be interpreted in the context of the given experimental setup, rather than as absolute, architecture-independent properties.

The overall experimental design is summarized in **Table 2.1**.

Table 2.1.: Overview of experimental configurations used to evaluate model performance. Baseline – evaluation under the default configuration ($\beta = 512$, $\sigma = 12$, 20 fine-tuning batches); FT – analysis of the impact of fine-tuning under a fixed beam width ($\beta = 512$); Joint – investigation of the joint effect of fine-tuning and beam width tuning.

Fine-tuning batches	Beam width		
	$\beta = 512$	$\beta = 1024$	$\beta = 2048$
20	Baseline + FT + Joint	Joint	Joint
40	FT + Joint	Joint	Joint
100	FT + Joint	Joint	Joint
Whole dataset	FT + Joint	Joint	Joint

It is important to note that computational efficiency metrics were collected concurrently with the performance experiments and therefore do not constitute a separate set of experiments. As a result, they are not explicitly reflected in **Table 2.1**.

2.9. Runtime and Computational Budget

All experiments are executed with a fixed wall-clock time budget of eight hours per run on a computing cluster equipped with four NVIDIA H100 GPUs (80 GB each). Individual runs are performed on a single GPU. Individual runs may be executed concurrently on the same GPU (up to two per device), depending on memory requirements. In total, this results in 2048 GPU-hours of computation (**Table 2.2**).

Table 2.2.: Overview of experimental configurations and computational budget. Baseline – evaluation under the default configuration ($\beta = 512$, $\sigma = 12$, 20 fine-tuning batches); FT – analysis of the impact of fine-tuning under a fixed beam width ($\beta = 512$); Joint – investigation of the joint effect of fine-tuning and beam width tuning.

Experimental setting	Baseline	FT	Joint
Number of benchmarks	20	8	4
Experiments per benchmark	1	4	8
Total number of experiments	20	32	24
Number of models evaluated	4	4	2
Runtime per experiment (hours)		8	
Compute hours per setting	640	1024	384
Total computational cost (GPU hours)		2048	

3. Results and Discussion

3.1. Pretraining Dynamics and Convergence Behavior

The training dynamics of all models are illustrated in **Figure 3.1**. Overall, the loss curves exhibit stable convergence behavior for all configurations, with similar trends observed across different batch sizes, indicating that the training dynamics are primarily determined by the model architecture rather than the batch size. This suggests that increasing the batch size from 256 to 512 does not significantly affect convergence dynamics or final loss values.

Notably, the xLSTM-based models consistently achieve lower training and validation losses compared to the transformer-based models, suggesting a more effective fit to the training distribution. At the same time, the transformer-based models exhibit a smaller gap between training and validation losses in the later stages of training, indicating more stable generalization behavior.

Furthermore, towards the later stages of training (around epoch 1000), the xLSTM-based models approach a plateau, with only marginal improvements in loss values. In contrast, the transformer-based models continue to exhibit a gradual decrease in loss, suggesting that it may benefit from extended training.

The evolution of the generalization gap also differs between the two architectures. For the xLSTM-based models, the gap between training and validation losses is initially small but gradually increases as training progresses, indicating a mild tendency toward overfitting. In contrast, the transformer-based models exhibit a larger gap in the early stages of training, which decreases over time and becomes comparatively smaller by the end of training. This highlights a trade-off between optimization efficiency and generalization, where xLSTM-based models achieve lower loss values, while the transformer-based models exhibit more stable generalization behavior.

Taken together, these observations suggest that while both architectures demonstrate stable training behavior across different batch sizes, the transformer-based models exhibit more robust generalization properties, whereas xLSTM-based models achieve a stronger fit to the training data at the cost of slightly increased overfitting. These differences in training behavior may influence downstream performance during fine-tuning and are further analyzed in the subsequent sections.

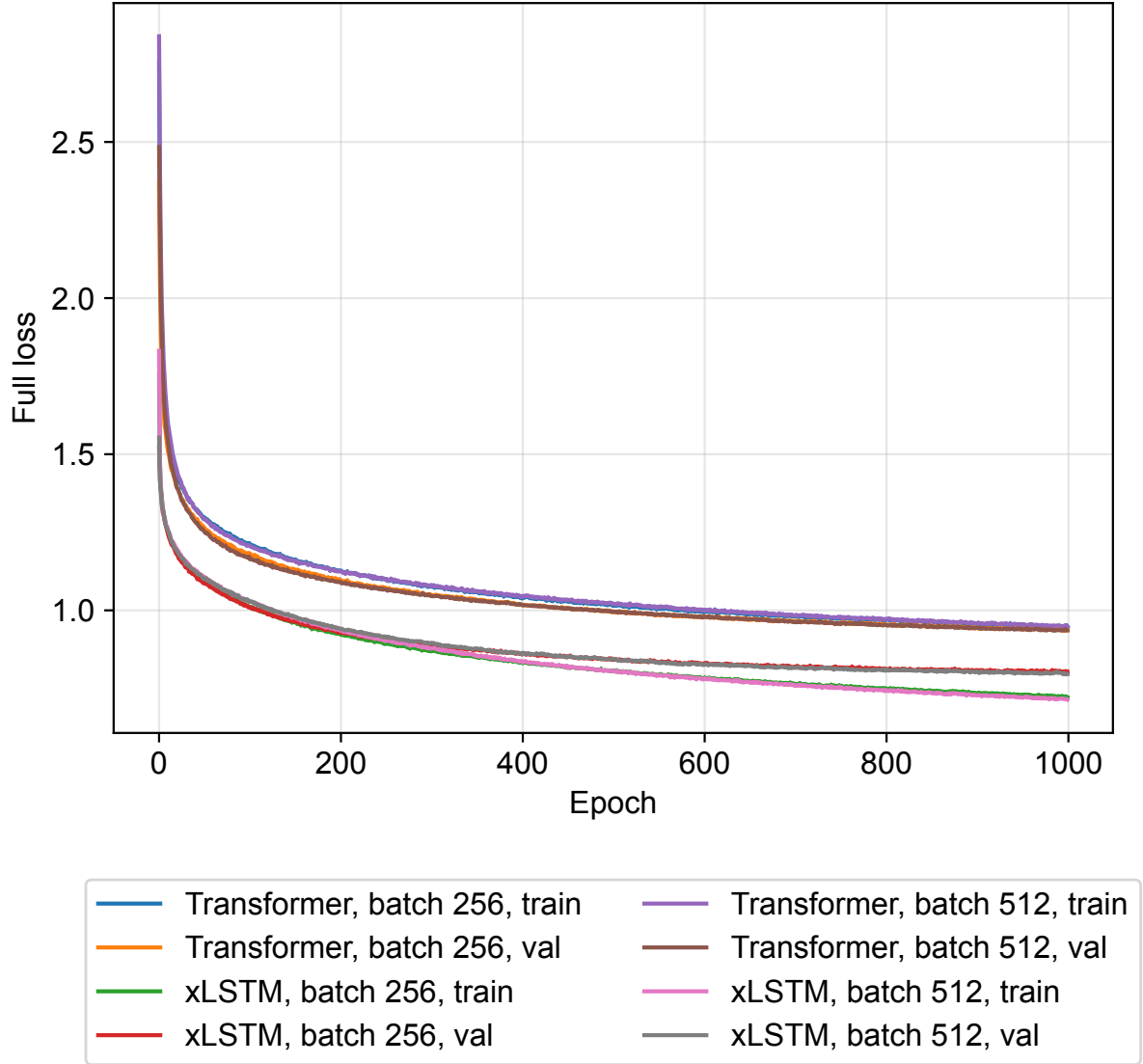


Figure 3.1.: Training and validation loss curves for transformer-based and xLSTM-based models under different batch size configurations during pretraining

Future work could explore extending pretraining to a larger number of epochs, particularly for transformer-based models, which continue to improve during later stages of training. Additionally, it would be valuable to investigate long-term training effects, such as delayed generalization or grokking phenomena [45, 46], which may further influence the relative performance of the considered architectures.

3.2. Evaluation of Model Performance under the Baseline Configuration

We begin by analyzing model performance under the baseline configuration, providing a unified reference point for subsequent experiments. The overall results are summarized in **Figure 3.2**, and detailed per-task scores are reported in **Table A.1**.

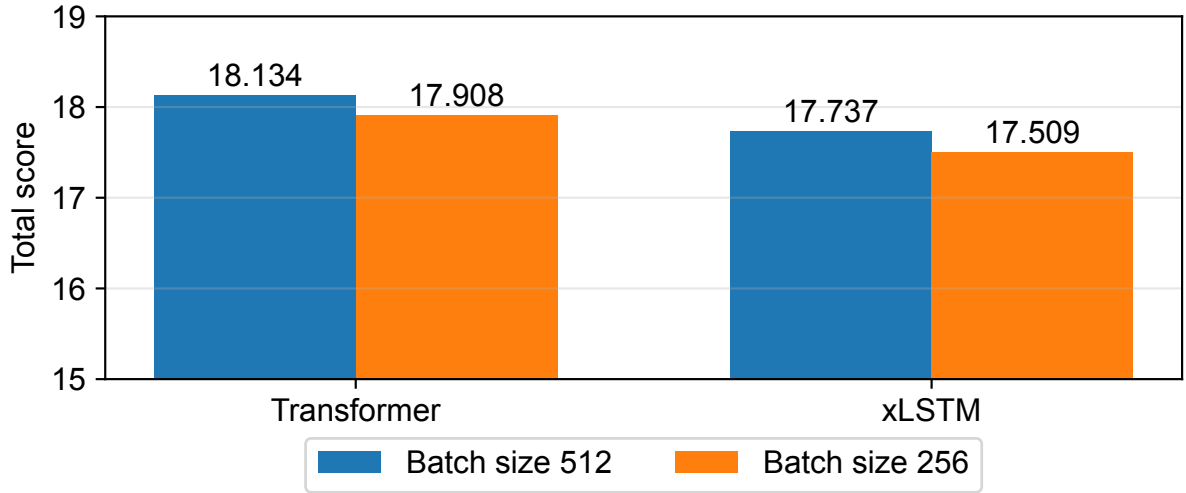


Figure 3.2.: Baseline performance of transformer-based and xLSTM-based models

A preliminary analysis reveals two consistent trends. First, transformer-based models outperform xLSTM-based models across the majority of tasks. In particular, the transformer-based model with batch size 512 achieves the highest total score of 18.134, followed by transformer 256 (17.908), xLSTM 512 (17.737), and xLSTM 256 (17.509). Second, models pretrained with a larger batch size (512) consistently achieve better performance than their counterparts trained with batch size 256.

Beyond aggregate performance, the transformer 512 configuration also attains the highest score on the largest number of individual benchmarks, indicating that its advantage is not limited to overall score but is consistently reflected across tasks (**Table A.1**). These observations provide an initial reference point for the subsequent analysis and motivate a more detailed investigation of architectural and training-related factors.

To facilitate a structured analysis of model behavior, benchmark tasks are grouped into three categories of increasing difficulty: easy, medium, and hard. This categorization is determined empirically based on performance under the baseline configuration.

Tasks for which all models consistently achieve the maximum score (1.0) are classified as easy. Tasks where only a subset of models reaches the maximum score, or where all models exhibit slightly lower yet still competitive performance (approximately 0.8–1.0), are categorized

as medium. Finally, tasks with substantially lower performance across all models are considered hard (**Table 3.1**).

Table 3.1.: Empirical grouping of GuacaMol benchmark tasks by difficulty under the baseline configuration.

Difficulty group	Benchmark tasks
Easy	Celecoxib rediscovery; Troglitazone rediscovery; Aripiprazole similarity; Albuterol similarity; Mestranol similarity; C11H24; Deco hop; C9H10N2O2PF2Cl
Medium	Thiothixene rediscovery; Osimertinib MPO; Fexofenadine MPO; Ranolazine MPO; Amlodipine MPO; Sitagliptin MPO; Valsartan SMARTS; Scaffold hop
Hard	Median molecules 1; Median molecules 2; Perindopril MPO; Zaleplon MPO

This grouping allows us to analyze model behavior under varying levels of difficulty and to better understand the impact of architectural and hyperparameter choices across different regimes.

Easy tasks do not provide discriminative power for comparing model performance, as all architectures consistently achieve the best possible scores. Instead, they serve as a sanity check for the overall framework. Importantly, the fact that optimal performance is achieved with minimal fine-tuning suggests that the pretrained models already capture the structural patterns required to solve these tasks. In this setting, fine-tuning primarily adjusts the output layer, while the learned representations remain largely sufficient.

The easy task category includes problems such as rediscovery, similarity optimization, and simple structure generation. These tasks primarily evaluate the ability of the models to reproduce known molecular patterns rather than to explore complex chemical spaces.

In contrast, medium and hard tasks provide a more informative evaluation setting. Medium tasks are used in the subsequent experiments in this work to analyze the impact of fine-tuning under a fixed beam width, while hard tasks, being the most challenging, are used to investigate the joint effect of fine-tuning and beam width tuning, as their performance is more sensitive to exploration strategies.

Focusing on medium and hard tasks allows for a more efficient and informative evaluation, avoiding redundant experiments on tasks where all models already achieve near-optimal performance.

Finally, we observe differences in the dynamics of reaching optimal solutions across tasks. While some tasks (e.g., Mestranol similarity, C11H24) are solved immediately and maintain the optimal score throughout training, others exhibit repeated rediscovery of optimal molecules. In contrast, certain tasks (e.g., Celecoxib rediscovery) reach the optimal solution only rarely, indicating a more challenging optimization landscape.

These observations suggest that task difficulty is not solely determined by the final score but also by the dynamics of discovering optimal solutions. Future work could therefore investigate task difficulty more systematically by analyzing how quickly and how consistently optimal scores are reached across different tasks and model configurations.

3.3. Impact of Fine-tuning under a Fixed Beam Width

In this section, we analyze the effect of fine-tuning intensity on model performance under a fixed beam width of $\beta = 512$.

As discussed above, we restrict the analysis to medium and hard tasks, as they provide sufficient discriminative power for meaningful comparison between models.

The results are presented in **Table A.2** and summarized in **Figure 3.3**, which shows the dependence of the total score on the number of fine-tuning batches per epoch. Overall, the two architectures exhibit notably different behavior.

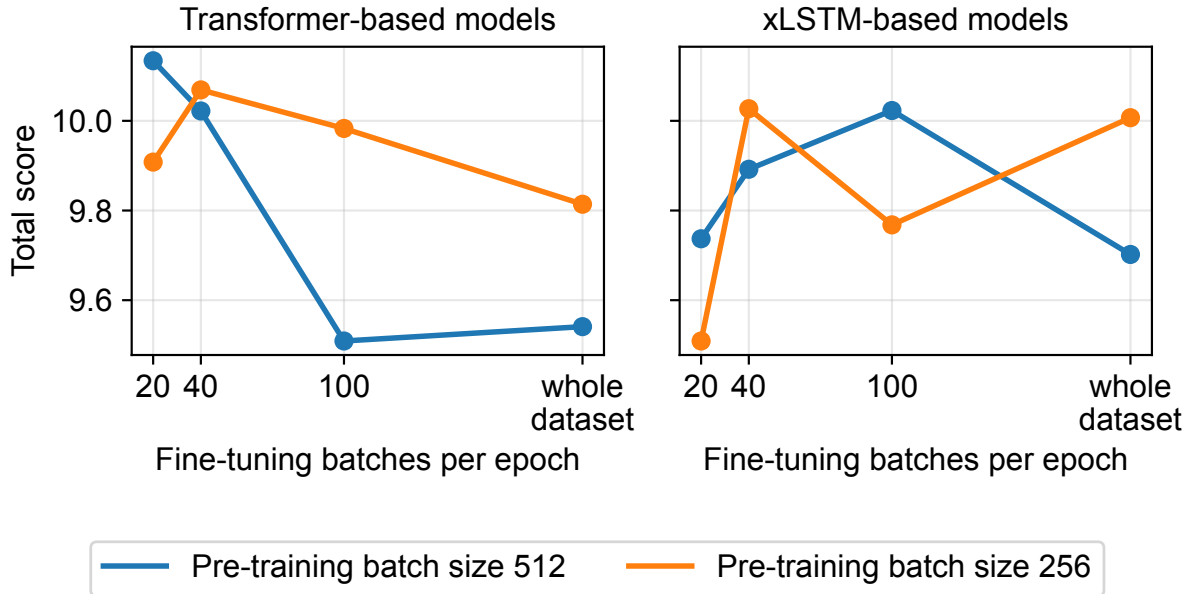


Figure 3.3.: Impact of fine-tuning intensity on transformer-based and xLSTM-based models ($\beta = 512$)

For transformer-based models, the best performance is achieved with a relatively small number of fine-tuning batches. In particular, the model pretrained with batch size 512 reaches its maximum already at 20 batches, while the model pretrained with batch size 256 peaks at 40 batches. Increasing the number of fine-tuning updates beyond these values does not lead to further improvements and even degrades performance. This behavior suggests that transformer-based models rely less on extensive fine-tuning and therefore tend toward lower levels of exploitation.

In contrast, xLSTM-based models tend to benefit from a more extended fine-tuning stage. The xLSTM-based model pretrained with batch size 512 achieves its best result at 100 batches, whereas the model pretrained with batch size 256 shows strong performance both at 40 batches and when performing a full pass over the entire BESTFOUND set. This indicates that xLSTM-based models rely more heavily on fine-tuning and thus tend toward higher levels of exploitation, with performance being more sensitive to the degree of adaptation applied during training.

More generally, increasing the number of fine-tuning batches shifts the balance toward stronger exploitation, while smaller numbers of batches correspond to a more exploration-oriented regime. This highlights that model performance is governed by a trade-off between exploration and exploitation, which is handled differently by the two architectures.

To enable a fair comparison, we consider the best observed performance for each model across all evaluated fine-tuning configurations (**Table A.3**). Under this evaluation, the transformer-based model pretrained with batch size 512 achieves the highest total score of 10.134, followed by the transformer-based model with batch size 256 (10.069). The best-performing xLSTM-based configurations achieve comparable but slightly lower results, with scores of 10.027 (batch size 256) and 10.023 (batch size 512).

Overall, while transformer-based models attain the highest peak performance under fixed configurations, the gap between the two architectures becomes relatively small when each model is evaluated under its optimal fine-tuning setup. This highlights the importance of properly tuning the fine-tuning process.

At the same time, the relationship between fine-tuning intensity and performance is not uniform across tasks. In several cases, performance exhibits behavior that deviates from the general trend. For example, in the Osimertinib MPO task, all models achieve optimal performance at 40 fine-tuning batches. However, this configuration is optimal only at the task level, while from the perspective of total score across all tasks it is optimal only for the transformer-based model pretrained with batch size 256.

Similar task-dependent behavior, where the optimal configuration for an individual task may differ from the model’s overall optimal setting, is observed in several other benchmarks (e.g., Fexofenadine MPO, Perindopril Rings, Ranolazine MPO), indicating that the optimal fine-tuning intensity depends both on the model architecture and the specific task.

These observations suggest that the trade-off between exploration and exploitation is not only model-dependent but also task-dependent. Different tasks may require different levels of fine-tuning intensity to achieve optimal performance, and no single global configuration is sufficient across all settings.

To further investigate this effect, we consider an oracle-like selection where, for each model and each task, the best result across all fine-tuning configurations is selected (**Table A.3**). Under this evaluation, the xLSTM-based model pretrained with batch size 512 achieves the highest

total score (10.219), outperforming both transformer-based models (10.205 and 10.179) as well as the xLSTM-based model pretrained with batch size 256 (10.125).

This result highlights that, although transformer-based models demonstrate stronger performance under fixed configurations, xLSTM-based models exhibit greater potential when the fine-tuning process is adapted to individual tasks.

Finally, it is important to note that the TASAR-based search process is inherently stochastic. As a result, part of the observed variability in performance may arise from stochastic effects rather than systematic differences between configurations. Future work should therefore include multiple runs per configuration to obtain statistically robust estimates (e.g., mean and variance) and to better distinguish between consistent trends and stochastic fluctuations.

3.4. Impact of the Joint Effect of Fine-tuning and Beam Width Tuning

In the previous experiment, where we analyzed the impact of fine-tuning intensity under a fixed beam width, hard tasks consistently exhibited the lowest performance across all models. Moreover, varying the number of fine-tuning batches did not lead to substantial or consistent improvements, with performance fluctuating within a relatively narrow range. This suggests that optimization on hard tasks is inherently unstable and may be limited by insufficient exploration during molecule generation.

Motivated by these observations, we hypothesize that improving performance on hard tasks requires increasing the level of exploration. To test this hypothesis, we extend the experimental setup by considering larger beam widths.

In addition, we introduce several adjustments tailored to the characteristics of the considered tasks. In particular, we relax constraints on molecule length to better accommodate tasks involving larger target structures (e.g., Perindopril Rings), and increase the frequency of intermediate reconsideration steps to improve local exploration in tasks involving shorter molecules (e.g., median molecule benchmarks). These modifications are designed to enable both broader exploration of the search space and more effective refinement of candidate solutions.

The following experiments are conducted using the transformer-based and xLSTM-based models pretrained with a batch size of 512, as these represent the best-performing configurations within each model family.

Firstly, we examine the overall effect of increasing the beam width on model performance. While certain local patterns can be observed for specific configurations, these trends are not consistent across the full set of experiments (**Figure 3.4**).

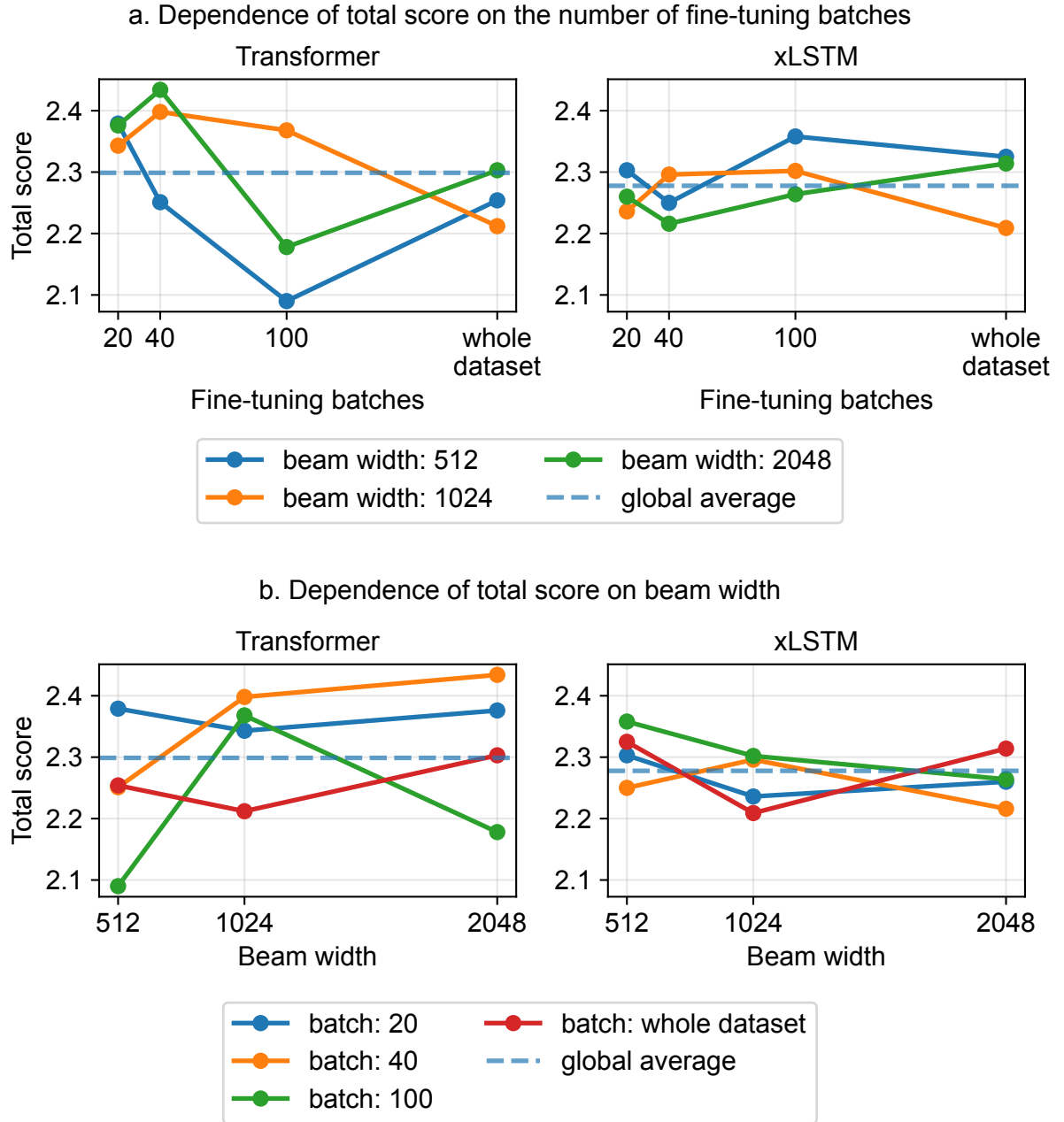


Figure 3.4.: Joint effect of fine-tuning intensity and beam width on model performance for hard tasks

For instance, in the transformer-based model with 40 fine-tuning batches (**Figure 3.4.b**), performance appears to increase monotonically with beam width, suggesting that additional exploration may be beneficial in this particular setting. However, this behavior is not reproduced under other configurations. For 20 batches and full-dataset fine-tuning, the performance follows a non-monotonic pattern, while the configuration with 100 batches exhibits an opposite trend, with performance peaking at a beam width $\beta = 1024$ and decreasing thereafter.

A similar pattern is observed across both architectures and all considered configurations. In particular, the behavior remains inconsistent: while some settings exhibit locally monotonic trends, others display non-monotonic dynamics, and no unified pattern can be identified.

Overall, both models appear to exhibit fluctuations around a model-specific average performance level. However, the nature of these fluctuations differs between architectures. The transformer-based model shows higher variability, reaching both the highest and the lowest total scores across configurations. In contrast, the xLSTM-based model demonstrates more stable behavior, with performance confined to a narrower range.

To further investigate this behavior, we consider the hypothesis that the observed fluctuations are primarily driven by the stochastic nature of the search process, rather than by systematic effects of hyperparameter tuning: given the limited number of evaluated configurations, the absence of repeated runs for each setting, and the lack of consistent trends across configurations, the observed patterns are unlikely to reflect a stable underlying relationship.

Under this assumption, model performance can be analyzed in terms of its overall mean and variability across configurations.

Accordingly, we compute the mean and standard deviation of the total score for each architecture across all considered settings (**Figure 3.5**). The transformer-based model achieves an average score of 2.299 with a standard deviation of 0.099, while the xLSTM-based model attains an average score of 2.278 with a standard deviation of 0.044.

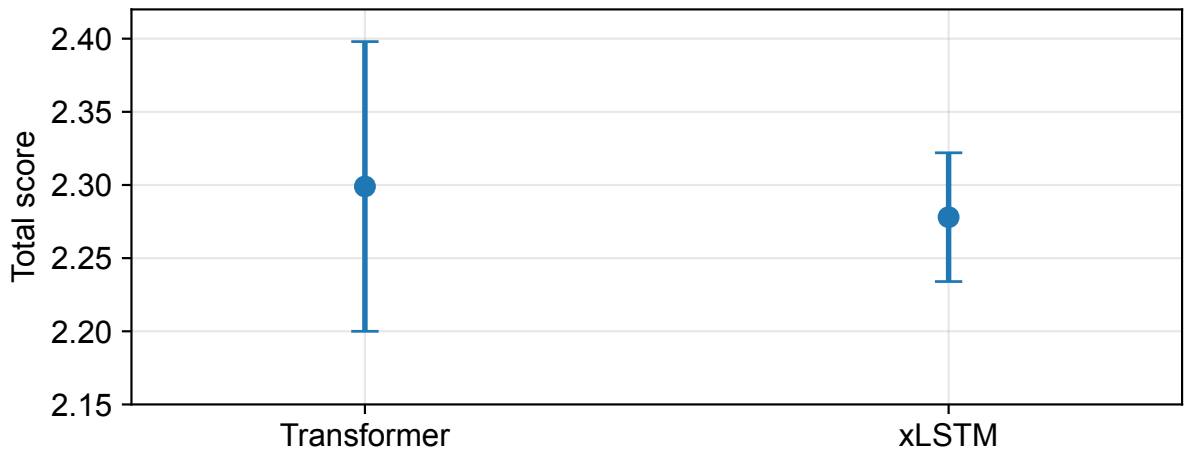


Figure 3.5.: Mean total score and standard deviation across all configurations for transformer-based and xLSTM-based models on hard tasks. Error bars indicate one standard deviation around the mean

The results indicate that both models exhibit fluctuations around a model-specific average performance level. Notably, the difference between the mean scores of the two architectures is small relative to their standard deviation, with the corresponding intervals largely overlapping.

This suggests that the observed difference in average performance is not substantial when compared to the inherent variability of the optimization process, and that both models achieve comparable performance under the considered settings.

On the other hand, a more structured pattern can be observed for the transformer-based model. In particular, its performance tends to improve with increasing beam width when combined with a moderate number of fine-tuning batches.

For example, at beam width $\beta = 512$ with 20 fine-tuning batches, the transformer-based model achieves a total score of 2.379. Increasing the beam width to $\beta = 1024$ and the number of fine-tuning batches to 40 yields an improved score of 2.398. Further increasing the beam width to $\beta = 2048$ while maintaining 40 fine-tuning batches results in a total score of 2.434, which is the highest observed value across all configurations (**Table A.4**, **Table A.5**, **Table A.6**).

This progression suggests that transformer-based models may benefit from increased exploration when paired with a moderate level of exploitation. Notably, the highest observed value (2.434) lies outside the variability range estimated in the previous analysis, indicating that this improvement may not be solely attributable to stochastic fluctuations.

In addition, across all considered beam widths, the transformer-based model consistently achieves slightly higher total scores compared to the xLSTM-based model. Although the differences are relatively small, this observation further supports the tendency of the transformer-based model to perform more favorably under the evaluated settings.

In contrast, such a consistent trend is not observed for the xLSTM-based model. Its performance remains more irregular across configurations, with no clear pattern of improvement as beam width increases.

However, it is important to note that, given the limited number of evaluated configurations and the absence of repeated runs, it is not possible to conclusively determine whether the observed trend for the transformer-based model reflects a stable underlying effect or is partially influenced by stochastic variability.

To further contextualize these results, we compare the obtained performance on hard tasks with the baseline results reported in the original GuacaMol benchmark [20]. As shown in **Table 3.2**, the overall performance of the considered models is consistent with previously reported results for other generative approaches, with all methods achieving comparable scores on these particularly challenging tasks.

Under the baseline configuration, the transformer-based model achieves a total score of 2.379, which is comparable to the Graph-based Genetic Algorithm (2.384), one of the strongest search-based baselines. In contrast, the xLSTM-based model reaches a lower total score of 2.303, underperforming even the SMILES LSTM baseline (2.337). This suggests that, despite its architectural improvements, the xLSTM-based model does not achieve its best observed performance under the baseline configuration, which appeared to be suboptimal for this model.

Table 3.2.: Performance comparison on hard tasks between the proposed models and baseline methods from the GuacaMol [20]

Benchmark	SMILES LSTM	Graph GA	Baseline configuration		Best configuration		Oracle-like selection	
			Trans.	xLSTM	Trans.	xLSTM	Trans.	xLSTM
Median molecules 1	0.438	0.406	0.474	0.459	0.419	0.459	0.475	0.475
Median molecules 2	0.422	0.432	0.371	0.403	0.459	0.408	0.459	0.417
Perindopril MPO	0.808	0.792	0.800	0.789	0.800	0.735	0.800	0.789
Zaleplon MPO	0.669	0.754	0.734	0.652	0.756	0.756	0.773	0.756
Total score	2.337	2.384	2.379	2.303	2.434	2.358	2.507	2.437

A plausible explanation for this behavior is that the xLSTM-based model appears to be more sensitive to the choice of hyperparameters and the balance between exploration and exploitation.

Another possible factor is computational efficiency: a SMILES-based LSTM model may generate molecules faster, allowing it to explore a larger portion of the chemical space within the same time budget and thereby increasing the likelihood of discovering higher-scoring molecules.

However, this interpretation changes when hyperparameter tuning and oracle-like selection are taken into account. In this setting, the performance of all models improves, reaching its maximum under oracle-like selection, where the best-performing configuration is chosen individually for each task.

Under this setting, the xLSTM-based model achieves a total score of 2.437, ranking second among all considered models. The transformer-based model attains the highest overall score of 2.507.

Taken together, these results reinforce the earlier conclusion that performance on hard tasks is not solely determined by model architecture, but is also influenced by the interaction between the model and the optimization procedure. In particular, the improvement in performance when moving from the baseline configuration to the best configuration and further to oracle-like selection indicates that hyperparameter tuning can be beneficial to a certain extent.

At the same time, a substantial portion of the observed variability may be driven by the stochastic nature of the Gumbel-based search process. Consequently, part of the observed performance improvements may also be attributed to increased sampling of candidate solutions rather than to systematic effects of the optimization procedure itself.

This further suggests that, for models with an inherent stochastic component, relying on a single performance value obtained from a single run, such as the score of the best generated molecule, may not provide a reliable basis for comparison. Instead, a more robust evaluation protocol would involve multiple independent runs under identical configurations, with performance reported in terms of summary statistics such as the mean and standard deviation.

This would allow for a more accurate assessment of both expected performance and variability, and would reduce the risk of drawing conclusions from random fluctuations.

In this context, an important direction for future work is to perform repeated experiments for each configuration in order to obtain statistically reliable estimates and to assess whether the observed improvements are systematic or primarily driven by stochastic variation.

Finally, these findings indicate that improvements on hard tasks may require approaches beyond simple hyperparameter tuning. In particular, future research could focus on the development of task-specific strategies tailored to the structural properties of individual benchmarks, as well as on a deeper analysis of the underlying chemical constraints that may limit model performance on challenging tasks.

3.5. Computational Efficiency During Pretraining

Recurrent architectures are traditionally less efficient during training due to their sequential nature, as each step depends on the previous hidden state. This dependency prevents full parallelization across sequence positions, since the computation of each subsequent state requires the completion of the previous one. However, modern variants such as xLSTM mitigate this limitation by redesigning the memory mechanism, reducing strict sequential dependencies and enabling more efficient parallelization.

As a result, the xLSTM-based model achieves training times that are comparable to, and in our experiments slightly lower than, those of the transformer-based model, indicating competitive computational efficiency despite its recurrent structure (**Table 3.3**).

Table 3.3.: Computational efficiency during pretraining for transformer-based and xLSTM-based models.

Model	Train time	Val. time	Total time	Throughput
Transformer 512	~33.0 min	~1.8 min	~34.8 min	~1.52 it/s
Transformer 256	~31.6 min	~2.0 min	~33.6 min	~3.16 it/s
xLSTM 512	~27.3 min	~1.8 min	~29.1 min	~1.83 it/s
xLSTM 256	~27.6 min	~2.0 min	~29.6 min	~3.62 it/s

In terms of memory usage, increasing the batch size leads to a substantial rise in GPU memory consumption. Notably, both models exhibit comparable memory usage under the same batch size, indicating that memory consumption is primarily driven by the batch size rather than the underlying model architecture (**Figure 3.6**).

However, even for the largest considered configuration (batch size 512), the available GPU memory (80 GB) is not fully saturated, suggesting that further scaling may still be feasible.

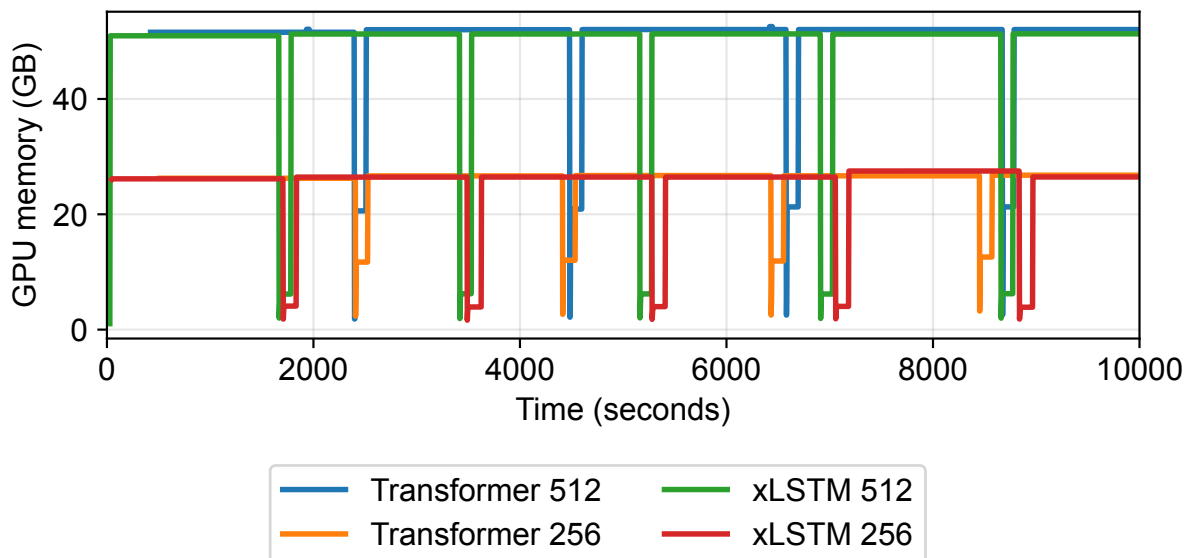


Figure 3.6.: GPU memory usage over time for transformer-based and xLSTM-based models under different batch size configurations

Finally, the choice of batch size involves a trade-off between computational efficiency and model performance. Reducing the batch size leads to higher throughput and lower GPU memory consumption, which can be beneficial when training on more constrained hardware. However, as observed in the previous experiments, smaller batch sizes tend to result in worse performance in molecule generation, particularly on medium-complexity tasks.

In contrast, larger batch sizes may improve generation quality, albeit at the cost of increased memory usage and slightly reduced throughput. Given that the available GPU memory (e.g., 80 GB on H100) is not fully utilized in our experiments, increasing the batch size could be a viable strategy to further improve performance, despite the potential increase in computational cost. In this regard, an important direction for future work is to explore larger batch sizes (e.g., 768 or higher) to assess their impact on generation quality and overall model performance.

3.6. Computational Efficiency During Generation

In our setup, each iteration of the learning algorithm consists of molecule generation, evaluation, and subsequent fine-tuning of the model based on the top-performing candidates. We refer to this full cycle as an optimization epoch.

The duration of such an epoch depends on several factors, including the model architecture, the complexity of the evaluation function, and, to some extent, the proportion of invalid molecules generated during sampling. As a result, within a fixed time budget (e.g., 8 hours), different benchmark tasks may allow for a significantly different number of optimization epochs, ranging

from tens to several hundreds. For example, this number can vary from roughly a dozen epochs for the Scaffold Hop task to several hundreds for Median Camphor-Menthol (**Table A.7**).

From an optimization perspective, a larger number of epochs implies that a greater portion of the chemical space can be explored, increasing the likelihood of discovering high-quality molecules. Therefore, the number of optimization epochs completed within a fixed time budget serves as a meaningful proxy for inference efficiency.

To obtain a comparable estimate across models, we aggregate the number of completed epochs over all benchmark tasks and use this value as a measure of overall generation efficiency.

To further analyze the effect of fine-tuning batch size on inference efficiency, we examine the dependence of the total number of completed optimization epochs on the number of fine-tuning batches. As shown in **Figure 3.7**, no clear or consistent pattern can be observed. Instead, the results exhibit fluctuations around a model-specific average level.

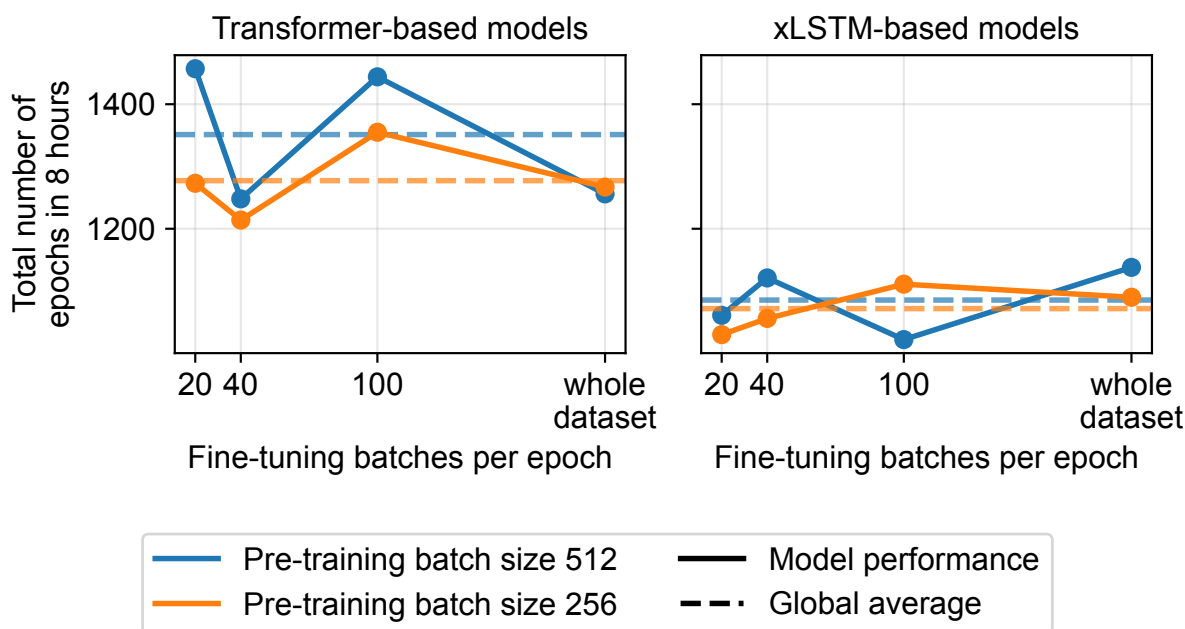


Figure 3.7.: Total number of optimization epochs completed within an 8-hour time budget as a function of fine-tuning batch size for transformer-based and xLSTM-based models

This behavior is consistent across both transformer-based and xLSTM-based models and suggests that the observed variations are primarily driven by stochastic effects inherent to the search procedure rather than by systematic differences in the fine-tuning configuration. Therefore, the average total number of completed epochs over all considered batch sizes, together with the corresponding standard deviation, provides a more robust and reliable measure of computational efficiency during generation.

Based on this aggregated metric (**Table 3.4**), the transformer-based models consistently achieve higher average numbers of completed epochs compared to the xLSTM-based models, indicating

superior inference efficiency. This difference is substantial and stable across configurations, suggesting that the transformer-based architecture is more computationally efficient for the considered generation procedure. Within each model family, the effect of the pretraining batch size is less pronounced. While models pretrained with a batch size of 512 tend to exhibit slightly higher efficiency than their counterparts trained with batch size 256, the gap remains relatively small, particularly for the xLSTM-based models.

Table 3.4.: Mean and standard deviation of the total number of completed optimization epochs within an 8-hour time budget for different model architectures and batch size configurations.

Summary statistics	Transformer 512	Transformer 256	xLSTM 512	xLSTM 256
Mean	1351	1277	1086	1072
Standard Deviation	115	58	54	36

In terms of variability, a consistent pattern can be observed across all configurations: models pretrained with a batch size of 512 exhibit higher standard deviation compared to those trained with batch size 256. Notably, the transformer-based model with batch size 512 shows the highest variability, while the remaining models display relatively similar and lower levels of variation.

One possible explanation for this behavior is that models exhibiting higher variability may explore a broader and more diverse portion of the chemical space. While such exploration can increase the likelihood of discovering high-quality molecules, it may also lead to less efficient sampling, for example due to a higher frequency of invalid or suboptimal candidates. This, in turn, could contribute to increased variability in the number of completed epochs across configurations.

Interestingly, despite the clear advantage of transformer-based models in terms of generation speed and the number of explored candidates, this does not translate into a proportional improvement in the final performance scores. As shown in the previous sections, both architectures achieve comparable results on the benchmark tasks.

This indicates that increasing the number of optimization epochs, and thus the extent of exploration of the chemical space, does not necessarily lead to better solutions. Instead, the effectiveness of individual search steps, i.e., the quality of generated candidates, appears to be a limiting factor. Consequently, computational efficiency in terms of speed alone is not sufficient to characterize the practical performance of generative models in this setting.

To complement the analysis of computational efficiency in terms of execution speed, we further examine memory usage during generation (**Figure 3.8**).

The results reveal a clear distinction between the two model architectures. First, xLSTM-based models are significantly more memory-efficient, consistently operating at lower memory levels throughout the generation process. In contrast, transformer-based models exhibit substantially

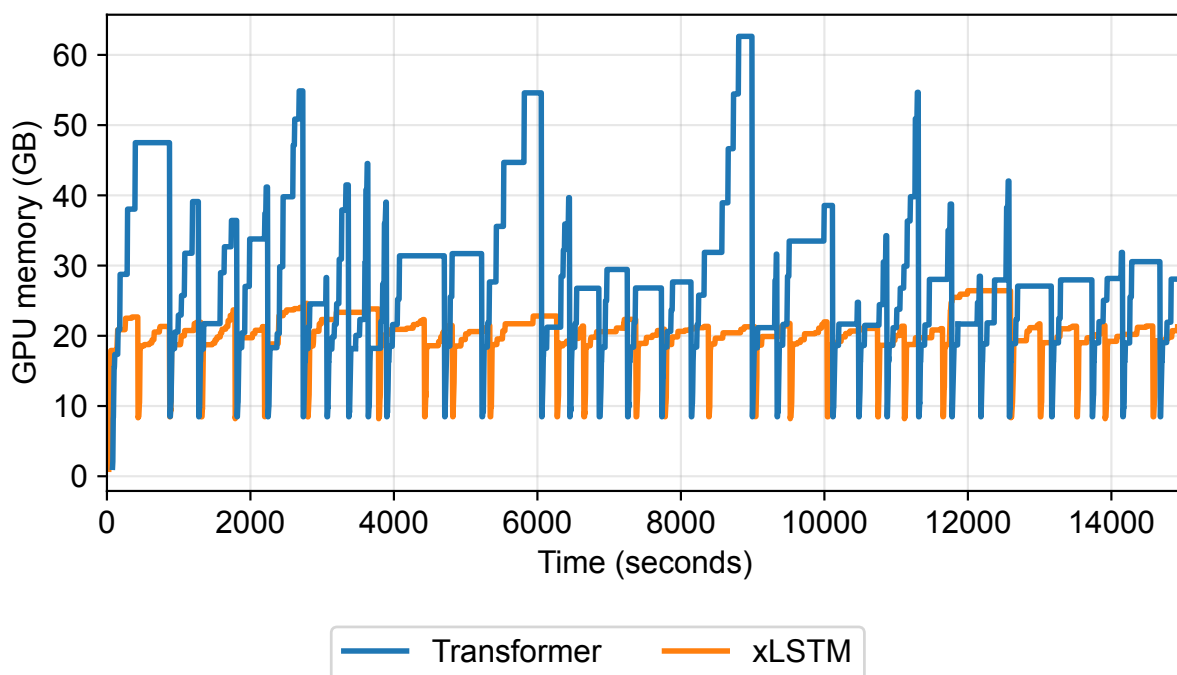


Figure 3.8.: GPU memory usage over time during molecule generation for transformer-based and xLSTM-based models

higher memory consumption, with peak usage reaching up to approximately three times higher under the same beam width.

This difference can be attributed to the quadratic complexity of the self-attention mechanism in transformer-based models with respect to sequence length. As molecule length increases, the number of pairwise interactions between tokens grows quadratically, requiring the storage of large intermediate attention maps. Consequently, memory consumption increases rapidly with sequence length. In contrast, xLSTM-based models rely on recurrent computations with linear complexity, leading to substantially lower and more predictable memory requirements. Furthermore, xLSTM-based models demonstrate more stable and uniform memory utilization over time, while transformer-based models exhibit pronounced fluctuations with sharp peaks. This suggests that xLSTM-based generation is not only more memory-efficient but also more consistent in its resource usage.

Importantly, the relatively short molecule lengths considered in this work (typically 50–70 atoms) may limit the observable advantages of xLSTM-based models in terms of overall computational efficiency. In this regime, highly optimized transformer implementations available in modern deep learning frameworks (e.g., PyTorch [47]) can partially offset their theoretical quadratic complexity, making the practical performance gap less pronounced. At the same time, xLSTM-based models may not fully benefit from similar levels of low-level optimization, which can further reduce the observable advantage of their linear computational complexity.

For longer sequences, such as polymers or proteins, where sequence length becomes substantially larger, the benefits of linear-complexity architectures are expected to become significantly more pronounced.

Taken together, these results highlight a discrepancy between computational efficiency and final model performance. While the considered architectures exhibit clear differences in the number of completed optimization epochs within a fixed time budget, their performance in terms of total score remains largely comparable. This suggests that the current evaluation setup may not sufficiently discriminate between models, as differences in efficiency do not translate into measurable differences in outcome quality.

As a direction for future work, it would be beneficial to consider alternative evaluation protocols that control for this effect. In particular, instead of fixing the total runtime, one could fix the number of optimization epochs and compare models under equal computational budgets in terms of performed updates. This would allow for a more direct assessment of the effectiveness of individual optimization steps and could provide a clearer comparison of model performance.

3.7. Effect of Beam Width on Computational Efficiency

As shown in the previous sections, increasing the beam width does not lead to consistent improvements in model performance. However, it has a substantial impact on computational efficiency.

In particular, increasing the beam width significantly reduces the number of optimization epochs that can be completed within a fixed time budget (**Figure 3.9**, **Table A.8**, **Table A.9**, **Table A.10**).

This behavior is consistent across both transformer-based and xLSTM-based models. As the beam width grows, each generation step involves exploring a larger number of candidate molecules, which increases the computational cost of sampling and evaluation. Consequently, each optimization epoch becomes more expensive, leading to a lower total number of completed epochs within the same time budget.

In addition to increased computational time, larger beam widths also result in higher GPU memory consumption (**Figure 3.10**).

This is due to the fact that beam search requires maintaining multiple candidate sequences simultaneously. For each beam, intermediate representations, model states, and partially generated molecules must be stored in memory, leading to an approximately linear increase in memory usage with respect to the beam width.

This effect is particularly pronounced for transformer-based models, where memory consumption is further amplified by the attention mechanism, which has quadratic complexity with respect to the sequence length. Since attention states must be maintained for each candidate in the beam,

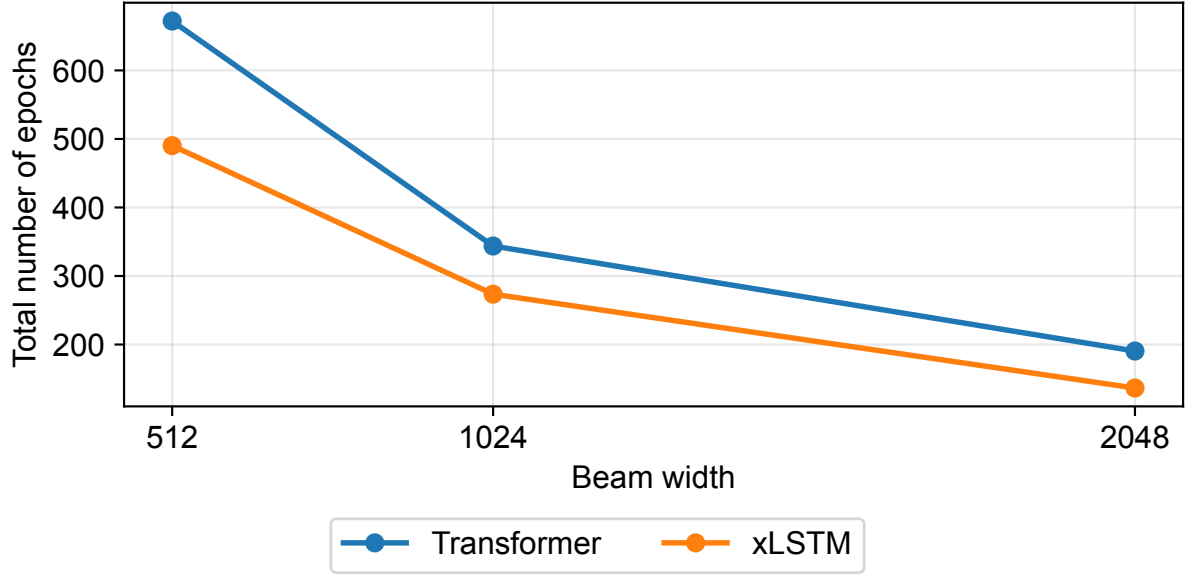


Figure 3.9.: Total number of optimization epochs completed within a fixed 8-hour time budget as a function of beam width for transformer-based and xLSTM-based models

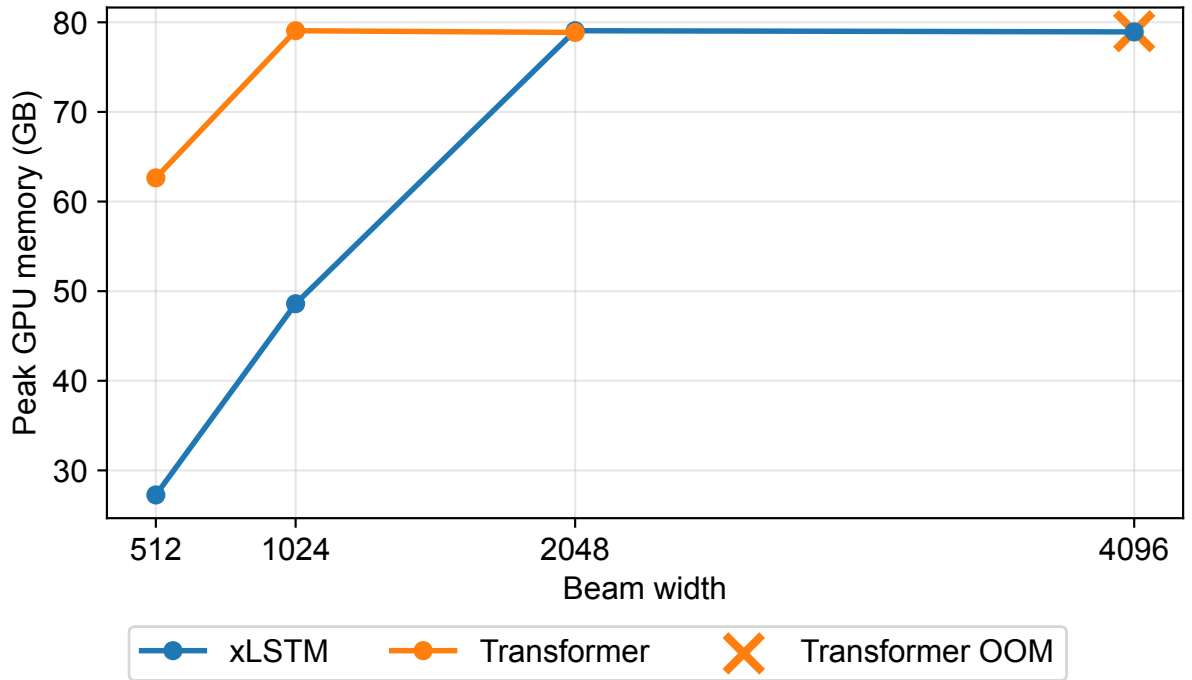


Figure 3.10.: Peak GPU memory usage as a function of beam width for transformer-based and xLSTM-based models. OOM denotes out-of-memory error

this leads to a rapid increase in memory usage as the beam width grows. In contrast, xLSTM-based models rely on recurrent-style computations with approximately linear scaling in sequence length, resulting in a more memory-efficient behavior. As a result, the transformer-based model reaches the GPU memory limit at a beam width of 4096, leading to an out-of-memory (OOM)

failure. On the other hand, the xLSTM-based model remains within memory limits under the same conditions, indicating more efficient memory usage.

This difference suggests that xLSTM-based models may be more suitable for scenarios requiring large-scale exploration with high beam widths, as they allow for a broader search of the chemical space under fixed hardware constraints. Taken together, these results highlight a trade-off between computational speed and memory efficiency. While transformer-based models enable faster exploration by completing more optimization epochs within the same time budget, xLSTM-based models provide better scalability with respect to beam width.

From a practical perspective, this suggests that model comparison under a fixed time budget alone (as it was suggested in the previous section) may not fully capture their relative advantages, as architectures differ in how efficiently they utilize available hardware resources. As a direction for future work, it would be informative to consider an evaluation protocol that jointly constrains both the number of optimization epochs and the maximum feasible beam width for a given hardware setup. In particular, fixing the largest beam width that each model can sustain without exceeding memory limits, and comparing performance under these conditions, would provide a more realistic assessment of their capabilities in large-scale exploration scenarios.

4. Conclusion

In this work, we compared transformer-based and xLSTM-based architectures as policy networks within the GraphXform workflow for molecular generation. The evaluation covered model performance across GuacaMol benchmark tasks, the impact of fine-tuning and beam search, and the computational efficiency of both architectures.

The main contribution of this work lies in extending the GraphXform framework by introducing xLSTM-based policy networks and providing a systematic empirical comparison with transformer-based architectures. This includes the implementation and pretraining of xLSTM-based models, their integration into the molecular generation workflow, and a comprehensive evaluation across multiple dimensions, including performance, and computational efficiency. Additionally, this work provides an in-depth analysis of the impact of key hyperparameters, such as beam width and fine-tuning configuration, and highlights the role of stochastic effects in the optimization process.

The results show that, under the baseline configuration, both architectures achieve comparable performance on medium and hard tasks, while easy tasks provide limited discriminative power. Transformer-based models exhibit a slight advantage in this setting. Further analysis of fine-tuning under a fixed beam width reveals that model performance is governed by a balance between exploration and exploitation, which is both model-dependent and task-dependent. Importantly, no single fine-tuning configuration consistently outperforms others, and the observed variability suggests a strong influence of the stochastic nature of the search process.

Extending the analysis to the joint effect of fine-tuning and beam width tuning, we find that increasing the beam width significantly raises computational cost while reducing the number of completed optimization epochs. However, this increased exploration does not lead to consistent improvements in performance. Instead, the results exhibit high variability across configurations, with no systematic gains observed, particularly on hard tasks.

The analysis of computational efficiency further highlights a key discrepancy between efficiency and performance. While transformer-based models are approximately 25% faster during generation and complete more optimization epochs within a fixed time budget, this advantage does not translate into better generation quality. In contrast, xLSTM-based models demonstrate significantly better memory efficiency and more stable resource utilization, while transformer-based models exhibit higher peak memory consumption and may reach hardware limits at large beam widths.

Taken together, these findings indicate that computational efficiency alone is not sufficient to characterize the practical performance of generative models in this setting. Instead, the effectiveness of individual search steps and the interaction between model architecture and the optimization process play a more critical role. Despite their comparable performance, transformer-based models appear more suitable as policy networks within the GraphXform workflow due to their simpler architecture, better alignment with molecular graph structure, and reduced sensitivity to hyperparameter tuning. At the same time, xLSTM-based models offer advantages in memory efficiency, which may become increasingly important in large-scale exploration scenarios under hardware constraints. Notably, the relatively short molecule lengths considered in this work (typically 50–70 atoms) may limit the observable benefits of xLSTM-based models; for longer sequences, such as polymers or proteins, its advantages are expected to become more pronounced.

Future work should focus on improving both evaluation methodology and model performance. In particular, it would be beneficial to extend pretraining to a larger number of epochs and explore larger batch sizes to further improve generation quality. Given the high variability of results, performing multiple runs per configuration and reporting summary statistics is essential for obtaining statistically robust conclusions. It is also important to validate whether the observed trends are consistent or primarily driven by stochastic effects. Additionally, task-specific strategies may be required to improve performance on hard benchmarks. Finally, alternative evaluation protocols that jointly constrain the number of optimization epochs and the maximum feasible beam width for a given hardware setup should be considered, as they may provide a more realistic and fair comparison of model capabilities.

Bibliography

- [1] G. Schneider and U. Fechner. “Computer-based de novo design of drug-like molecules”. In: *Nature reviews Drug discovery* 4.8 (2005), pp. 649–663.
- [2] D. C. Elton, Z. Boukouvalas, M. D. Fuge, and P. W. Chung. “Deep learning for molecular design—a review of the state of the art”. In: *Molecular Systems Design & Engineering* 4.4 (2019), pp. 828–849.
- [3] R. Gómez-Bombarelli, J. N. Wei, D. Duvenaud, J. M. Hernández-Lobato, B. Sánchez-Lengeling, D. Sheberla, J. Aguilera-Iparraguirre, T. D. Hirzel, R. P. Adams, and A. Aspuru-Guzik. “Automatic chemical design using a data-driven continuous representation of molecules”. In: *ACS central science* 4.2 (2018), pp. 268–276.
- [4] D. Weininger. “SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules”. In: *Journal of chemical information and computer sciences* 28.1 (1988), pp. 31–36.
- [5] M. Krenn, F. Häse, A. Nigam, P. Friederich, and A. Aspuru-Guzik. “Self-referencing embedded strings (SELFIES): A 100% robust molecular string representation”. In: *Machine Learning: Science and Technology* 1.4 (2020), p. 045024.
- [6] W. Jin, R. Barzilay, and T. Jaakkola. “Junction tree variational autoencoder for molecular graph generation”. In: *International conference on machine learning*. PMLR. 2018, pp. 2323–2332.
- [7] Z. Zhou, S. Kearnes, L. Li, R. N. Zare, and P. Riley. “Optimization of molecules via deep reinforcement learning”. In: *Scientific reports* 9.1 (2019), p. 10752.
- [8] Y. Wang, Z. Li, and A. Barati Farimani. “Graph neural networks for molecules”. In: *Machine learning in molecular sciences*. Springer, 2023, pp. 21–66.
- [9] M. Olivecrona, T. Blaschke, O. Engkvist, and H. Chen. “Molecular de-novo design through deep reinforcement learning”. In: *Journal of cheminformatics* 9.1 (2017), p. 48.
- [10] M. H. Segler, T. Kogej, C. Tyrchan, and M. P. Waller. “Generating focused molecule libraries for drug discovery with recurrent neural networks”. In: *ACS central science* 4.1 (2018), pp. 120–131.
- [11] S. Chithrananda, G. Grand, and B. Ramsundar. “ChemBERTa: large-scale self-supervised pretraining for molecular property prediction”. In: *arXiv preprint arXiv:2010.09885* (2020).
- [12] N. O’Boyle and A. Dalke. “DeepSMILES: an adaptation of SMILES for use in machine-learning of chemical structures”. In: *ChemRxiv* (2018).

- [13] M. Krenn, F. Häse, A. Nigam, P. Friederich, and A. Aspuru-Guzik. “SELFIES: a robust representation of semantically constrained graphs with an example application in chemistry”. In: *arXiv preprint arXiv:1905.13741* 1.3 (2019).
- [14] H. Dai, Y. Tian, B. Dai, S. Skiena, and L. Song. “Syntax-directed variational autoencoder for structured data”. In: *arXiv preprint arXiv:1802.08786* (2018).
- [15] N. Yoshikawa, K. Terayama, M. Sumita, T. Homma, K. Oono, and K. Tsuda. “Population-based de novo molecule generation, using grammatical evolution”. In: *Chemistry Letters* 47.11 (2018), pp. 1431–1434.
- [16] J. H. Jensen. “A graph-based genetic algorithm and generative model/Monte Carlo tree search for the exploration of chemical space”. In: *Chemical science* 10.12 (2019), pp. 3567–3572.
- [17] P. Polishchuk. “CReM: chemically reasonable mutations framework for structure generation”. In: *Journal of Cheminformatics* 12.1 (2020), p. 28.
- [18] X. Hu, G. Liu, Y. Zhao, and H. Zhang. “De novo drug design using reinforcement learning with multiple gpt agents”. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 7405–7418.
- [19] S. Ahn, J. Kim, H. Lee, and J. Shin. “Guiding deep molecular optimization with genetic exploration”. In: *Advances in neural information processing systems* 33 (2020), pp. 12008–12021.
- [20] N. Brown, M. Fiscato, M. H. Segler, and A. C. Vaucher. “GuacaMol: benchmarking models for de novo molecular design”. In: *Journal of chemical information and modeling* 59.3 (2019), pp. 1096–1108.
- [21] J. Pirnay, J. G. Rittig, A. B. Wolf, M. Grohe, J. Burger, A. Mitsos, and D. G. Grimm. “GraphX-Form: graph transformer for computer-aided molecular design”. In: *Digital Discovery* 4.4 (2025), pp. 1052–1065.
- [22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [23] I. Beltagy, M. E. Peters, and A. Cohan. “Longformer: The long-document transformer”. In: *arXiv preprint arXiv:2004.05150* (2020).
- [24] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler. “Efficient transformers: A survey”. In: *ACM Computing Surveys* 55.6 (2022), pp. 1–28.
- [25] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter. “xlstm: Extended long short-term memory”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 107547–107603.
- [26] S. Hochreiter and J. Schmidhuber. “Long short-term memory”. In: *Neural computation* 9.8 (1997), pp. 1735–1780.
- [27] F. A. Gers, J. Schmidhuber, and F. Cummins. “Learning to forget: Continual prediction with LSTM”. In: *Neural computation* 12.10 (2000), pp. 2451–2471.

- [28] N. Schmidinger, L. Schneckenreiter, P. Seidl, J. Schimunek, P.-J. Hoedt, J. Brandstetter, A. Mayr, S. Luukkonen, S. Hochreiter, and G. Klambauer. “Bio-xLSTM: Generative modeling, representation and in-context learning of biological and chemical sequences”. In: *arXiv preprint arXiv:2411.04165* (2024).
- [29] Y. Sun, Y. Lu, Y. Y. Li, Z. Jing, C. K. Leung, and P. Hu. “MolGraph-xLSTM: A graph-based dual-level xLSTM framework with multi-head mixture-of-experts for enhanced molecular representation and interpretability”. In: *arXiv preprint arXiv:2501.18439* (2025).
- [30] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.
- [31] A. K. Vijayakumar, M. Cogswell, R. R. Selvaraju, Q. Sun, S. Lee, D. Crandall, and D. Batra. “Diverse beam search: Decoding diverse solutions from neural sequence models”. In: *arXiv preprint arXiv:1610.02424* (2016).
- [32] J. Pirnay and D. G. Grimm. “Take a step and reconsider: sequence decoding for self-improved neural combinatorial optimization”. In: *arXiv preprint arXiv:2407.17206* (2024).
- [33] W. Kool, H. Van Hoof, and M. Welling. “Stochastic beams and where to find them: The gumbel-top-k trick for sampling sequences without replacement”. In: *International conference on machine learning*. PMLR, 2019, pp. 3499–3508.
- [34] M. Freitag and Y. Al-Onaizan. “Beam Search Strategies for Neural Machine Translation”. In: *Proceedings of the First Workshop on Neural Machine Translation*. 2017, pp. 56–60.
- [35] C. Meister, A. Amini, T. Vieira, and R. Cotterell. *Conditional Poisson Stochastic Beam Search*. 2023. arXiv: 2109.11034 [cs.CL].
- [36] K. Shi, D. Bieber, and C. Sutton. *Incremental Sampling Without Replacement for Sequence Models*. 2021. arXiv: 2002.09067 [cs.LG].
- [37] A. Z. Wagner. *Constructions in combinatorics via neural networks*. 2021. arXiv: 2104.14516 [math.CO].
- [38] A. Corsini, A. Porrello, S. Calderara, and M. Dell’Amico. “Self-labeling the job shop scheduling problem”. In: *Advances in Neural Information Processing Systems 37* (2024), pp. 105528–105551.
- [39] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu. “Do Transformers Really Perform Badly for Graph Representation?” In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan. Vol. 34. Curran Associates, Inc., 2021, pp. 28877–28888.
- [40] T. Bachlechner, B. P. Majumder, H. Mao, G. Cottrell, and J. McAuley. “Rezero is all you need: Fast convergence at large depth”. In: *Uncertainty in artificial intelligence*. PMLR, 2021, pp. 1352–1361.
- [41] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention”. In: *International Conference on Machine Learning* (2020).

- [42] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. “Neural Message Passing for Quantum Chemistry”. In: *International Conference on Machine Learning*. 2017.
- [43] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-i. Kawarabayashi, and S. Jegelka. “Representation learning on graphs with jumping knowledge networks”. In: *International conference on machine learning*. pmlr. 2018, pp. 5453–5462.
- [44] M. Beck, K. Pöppel, P. Lippe, R. Kurle, P. M. Blies, G. Klambauer, S. Böck, and S. Hochreiter. “xLstm 7b: A recurrent llm for fast and efficient inference”. In: *arXiv preprint arXiv:2503.13427* (2025).
- [45] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. “Grokking: Generalization beyond overfitting on small algorithmic datasets”. In: *arXiv preprint arXiv:2201.02177* (2022).
- [46] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. “Progress measures for grokking via mechanistic interpretability”. In: *arXiv preprint arXiv:2301.05217* (2023).
- [47] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. “Pytorch: An imperative style, high-performance deep learning library”. In: *Advances in neural information processing systems* 32 (2019).

A. Appendix

Table A.1.: Empirical task categorization based on performance under the baseline configuration.
Bold values indicate the highest score for each task across all model configurations.

Benchmark	Transformer 512	Transformer 256	xLSTM 512	xLSTM 256
<i>Easy tasks</i>				
Celecoxib rediscovery	1.000	1.000	1.000	1.000
Troglitazone rediscovery	1.000	1.000	1.000	1.000
Aripiprazole similarity	1.000	1.000	1.000	1.000
Albuterol similarity	1.000	1.000	1.000	1.000
Mestranol similarity	1.000	1.000	1.000	1.000
C11H24*	1.000	1.000	1.000	1.000
Deco hop	1.000	1.000	1.000	1.000
C9H10N2O2PF2Cl†	1.000	1.000	1.000	1.000
<i>Medium tasks</i>				
Thiothixene Rediscovery	1.000	0.840	0.865	0.753
Osimertinib MPO	0.967	0.992	0.995	0.960
Fexofenadine MPO	1.000	0.999	0.927	0.925
Ranolazine MPO	0.951	0.897	0.887	0.866
Amlodipine MPO	0.897	0.878	0.894	0.827
Sitagliptin MPO	0.941	0.926	0.889	0.883
Valsartan SMARTS	0.999	0.995	0.997	0.998
Scaffold hop	1.000	1.000	0.980	1.000
<i>Hard tasks</i>				
Median molecules 1	0.474	0.459	0.459	0.460
Median molecules 2	0.371	0.413	0.403	0.413
Perindopril MPO	0.800	0.814	0.789	0.792
Zaleplon MPO	0.734	0.695	0.652	0.632
Total score	18.134	17.908	17.737	17.509

Table A.2.: Performance on medium and hard tasks under varying fine-tuning intensities with a fixed beam width ($\beta = 512$). Results are reported for all model configurations across different numbers of fine-tuning batches per epoch. T512 – transformer pretrained with batch size 512; T256 – transformer pretrained with batch size 256; L512 – xLSTM pretrained with batch size 512; L256 – xLSTM pretrained with batch size 256. Bold values indicate the highest score for a given task within a specific fine-tuning configuration, while highlighted cells denote the best result achieved across all configurations for that task.

Benchmark	Number of batches per epoch for fine-tuning																Whole dataset			
	20				40				100											
	T512	T256	L512	L256	T512	T256	L512	L256	T512	T256	L512	L256	T512	T256	L512	L256	T512	T256	L512	L256
Amlodipine MPO	0.897	0.878	0.894	0.827	0.908	0.878	0.894	0.894	0.908	0.894	0.886	0.883	0.835	0.906	0.906	0.906	0.878			
Fexofenadine MPO	1.000	0.999	0.927	0.925	1.000	0.999	0.967	0.989	0.944	1.000	1.000	0.953	1.000	1.000	0.961	0.950				
Osimertinib MPO	0.967	0.992	0.995	0.960	1.000	1.000	1.000	1.000	1.000	1.000	0.953	0.941	0.987	1.000	0.933	0.970				
Ranolazine MPO	0.951	0.897	0.887	0.866	0.933	0.881	0.880	0.874	0.933	0.908	0.897	0.875	0.913	0.891	0.957	0.918				
Scaffold hop	1.000	1.000	0.980	1.000	1.000	1.000	0.982	0.985	1.000	1.000	1.000	1.000	0.867	1.000	1.000	1.000				
Sitagliptin MPO	0.941	0.926	0.889	0.883	0.934	0.941	0.924	0.893	0.903	0.965	0.930	0.916	0.910	0.954	0.936	0.917				
Thiothixene Rediscovery	1.000	0.840	0.865	0.753	1.000	1.000	1.000	1.000	0.735	0.876	1.000	0.814	0.776	0.750	0.685	1.000				
Valsartan SMARTS	0.999	0.995	0.997	0.998	0.996	0.995	0.995	0.995	0.996	0.997	0.999	0.998	0.999	1.000	0.999	0.997				
Median Camphor-Menthol	0.474	0.459	0.459	0.460	0.475	0.459	0.459	0.474	0.475	0.476	0.459	0.474	0.475	0.459	0.459	0.474				
Median Tadalafil-Sildenafil	0.371	0.413	0.403	0.413	0.333	0.410	0.403	0.413	0.337	0.410	0.408	0.417	0.323	0.409	0.417	0.409				
Perindopril Rings	0.800	0.814	0.789	0.792	0.799	0.799	0.713	0.834	0.642	0.734	0.735	0.814	0.765	0.789	0.754	0.835				
Zaleplon MPO	0.734	0.695	0.652	0.632	0.644	0.707	0.675	0.676	0.636	0.723	0.756	0.683	0.691	0.656	0.695	0.659				
Total score	10.134	9.908	9.737	9.509	10.022	10.069	9.892	10.027	9.509	9.983	10.023	9.768	9.541	9.814	9.702	10.007				

Table A.3.: Comparison of model performance under fixed configuration and oracle-like selection. T512 – transformer pretrained with batch size 512; T256 – transformer pretrained with batch size 256; L512 – xLSTM pretrained with batch size 512; L256 – xLSTM pretrained with batch size 256. Bold values indicate the highest score for a given task within a specific selection setup, while highlighted cells denote the best result achieved across all setups for that task.

Benchmark	Best configuration selection						Oracle-like selection $\beta = 512$						Oracle-like selection $\beta = \{512, 1024, 2048\}$					
	T512	T256	L512	L256	T512	T256	L512	L256	T512	T256	L512	L256	T512	T256	L512	L256	T512	L512
Amlodipine MPO	0.897	0.878	0.886	0.894	0.908	0.906	0.894	0.894	0.908	0.906	0.906	0.894	0.908	0.906	0.906	0.894	0.908	0.906
Fexofenadine MPO	1.000	0.999	1.000	0.989	1.000	1.000	0.989	0.989	1.000	1.000	1.000	0.989	1.000	1.000	1.000	0.989	1.000	1.000
Osimertinib MPO	0.967	1.000	0.953	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Ranolazine MPO	0.951	0.881	0.897	0.874	0.951	0.908	0.874	0.951	0.951	0.908	0.957	0.918	0.951	0.957	0.957	0.918	0.951	0.957
Scaffold hop	1.000	1.000	1.000	0.985	1.000	1.000	0.985	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Sitagliptin MPO	0.941	0.941	0.930	0.893	0.941	0.965	0.893	0.941	0.941	0.965	0.936	0.917	0.941	0.936	0.936	0.917	0.941	0.936
Thiothixene Rediscovery	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Valsartan SMARTS	0.999	0.995	0.999	0.995	0.999	1.000	0.995	0.999	0.999	1.000	0.999	0.998	0.999	0.999	0.999	0.998	0.999	0.999
Median Camphor-Menthol	0.474	0.459	0.459	0.474	0.475	0.476	0.459	0.475	0.475	0.476	0.459	0.474	0.475	0.459	0.459	0.474	0.475	0.475
Median Tadalafil-Sildenafil	0.371	0.410	0.408	0.413	0.371	0.413	0.408	0.371	0.371	0.413	0.417	0.417	0.459	0.417	0.417	0.417	0.459	0.417
Perindopril Rings	0.800	0.799	0.735	0.834	0.800	0.814	0.735	0.800	0.800	0.814	0.789	0.835	0.800	0.789	0.789	0.835	0.800	0.789
Zaleplon MPO	0.734	0.707	0.756	0.676	0.734	0.723	0.676	0.734	0.734	0.723	0.756	0.683	0.773	0.756	0.756	0.683	0.773	0.756
Total score	10.134	10.069	10.023	10.027	10.179	10.205	10.023	10.027	10.179	10.205	10.219	10.125	10.306	10.219	10.306	10.125	10.306	10.235

Table A.4.: Performance on hard tasks as a function of fine-tuning intensity ($\beta = 512$). Bold values indicate the highest score for a given task within a specific fine-tuning configuration, while highlighted cells denote the best result achieved across all configurations for that task.

Benchmark	Number batches per epoch for fine-tuning							
	20		40		100		whole dataset	
	T512	L512	T512	L512	T512	L512	T512	L512
Median Camphor-Menthol	0.474	0.459	0.475	0.459	0.475	0.459	0.475	0.459
Median Tadalafil-Sildenafil	0.371	0.403	0.333	0.403	0.337	0.408	0.323	0.417
Perindopril Rings	0.800	0.789	0.799	0.713	0.642	0.735	0.765	0.754
Zaleplon MPO	0.734	0.652	0.644	0.675	0.636	0.756	0.691	0.695
Total score	2.379	2.303	2.251	2.250	2.090	2.358	2.254	2.325

Table A.5.: Performance on hard tasks as a function of fine-tuning intensity ($\beta = 1024$). Bold values indicate the highest score for a given task within a specific fine-tuning configuration, while highlighted cells denote the best result achieved across all configurations for that task.

Benchmark	Number batches per epoch for fine-tuning							
	20		40		100		whole dataset	
	T512	L512	T512	L512	T512	L512	T512	L512
Median Camphor-Menthol	0.475	0.459	0.459	0.459	0.475	0.459	0.459	0.475
Median Tadalafil-Sildenafil	0.392	0.387	0.390	0.409	0.394	0.370	0.337	0.335
Perindopril Rings	0.742	0.742	0.776	0.789	0.780	0.732	0.787	0.743
Zaleplon MPO	0.734	0.648	0.773	0.639	0.719	0.741	0.629	0.656
Total score	2.343	2.236	2.398	2.296	2.368	2.302	2.212	2.209

Table A.6.: Performance on hard tasks as a function of fine-tuning intensity ($\beta = 2048$). Bold values indicate the highest score for a given task within a specific fine-tuning configuration, while highlighted cells denote the best result achieved across all configurations for that task.

Benchmark	Number batches per epoch for fine-tuning							
	20		40		100		whole dataset	
	T512	L512	T512	L512	T512	L512	T512	L512
Median Camphor-Menthol	0.475	0.459	0.419	0.459	0.459	0.459	0.459	0.459
Median Tadalafil-Sildenafil	0.407	0.387	0.459	0.387	0.352	0.400	0.394	0.387
Perindopril Rings	0.779	0.723	0.800	0.695	0.723	0.734	0.727	0.741
Zaleplon MPO	0.715	0.691	0.756	0.675	0.644	0.671	0.723	0.727
Total score	2.376	2.260	2.434	2.216	2.178	2.264	2.303	2.314

Table A.7.: Number of training epochs completed within an 8-hour time budget across different fine-tuning configurations and fixed beam widths ($\beta = 512$). T512 – transformer pretrained with batch size 512; T256 – transformer pretrained with batch size 256; L512 – xLSTM pretrained with batch size 512; L256 – xLSTM pretrained with batch size 256.

Benchmark	Number batches per epoch for fine-tuning															
	20				40				100				Whole dataset			
	T512	T256	L512	L256	T512	T256	L512	L256	T512	T256	L512	L256	T512	T256	L512	L256
Amlodipine MPO	123	111	102	100	110	101	119	106	108	129	97	107	101	134	109	98
Fexofenadine MPO	89	81	66	67	66	84	78	64	81	75	58	72	85	77	66	60
Osimertinib MPO	76	70	65	61	55	73	60	63	57	58	64	65	51	59	53	58
Ranolazine MPO	74	59	59	57	69	61	63	55	70	73	60	55	71	56	58	55
Scaffold hop	18	20	15	15	14	17	12	11	6	19	11	10	22	20	11	14
Sitagliptin MPO	141	87	93	86	118	98	108	92	156	101	109	89	89	118	111	120
Thiothixene Rediscovery	91	131	96	89	98	110	87	88	101	119	83	81	96	105	103	88
Valsartan SMARTS	88	83	76	81	159	82	94	85	137	127	97	81	97	144	98	109
Median Camphor-Menthhol	384	350	253	234	260	325	244	246	433	366	212	262	308	263	274	241
Median Tadalafil-Sildenafil	77	84	72	70	74	71	64	73	58	62	57	70	73	62	56	63
Perindopril Rings	92	77	63	69	75	69	67	61	54	70	56	65	90	93	62	64
Zaleplon MPO	204	120	101	101	150	123	125	112	183	156	118	154	173	136	137	120
Total number	1457	1273	1061	1030	1248	1214	1121	1056	1444	1355	1022	1111	1256	1267	1138	1090

Table A.8.: Number of training epochs completed within an 8-hour time budget as a function of fine-tuning intensity ($\beta = 512$).

Benchmark	Number batches per epoch for fine-tuning							
	20		40		100		whole dataset	
	T512	L512	T512	L512	T512	L512	T512	L512
Median Camphor-Menthol	384	253	260	244	433	212	308	274
Median Tadalafil-Sildenafil	77	72	74	64	58	57	73	56
Perindopril Rings	92	63	75	67	54	56	90	62
Zaleplon MPO	204	101	150	125	183	118	173	137
Total number	757	489	559	500	728	443	644	529

Table A.9.: Number of training epochs completed within an 8-hour time budget as a function of fine-tuning intensity ($\beta = 1024$).

Benchmark	Number batches per epoch for fine-tuning							
	20		40		100		whole dataset	
	T512	L512	T512	L512	T512	L512	T512	L512
Median Camphor-Menthol	141	132	204	137	192	136	188	163
Median Tadalafil-Sildenafil	42	38	38	33	34	36	38	35
Perindopril Rings	46	36	34	33	31	36	36	38
Zaleplon MPO	64	52	96	56	88	64	103	69
Total number	293	258	372	259	345	272	365	305

Table A.10.: Number of training epochs completed within an 8-hour time budget as a function of fine-tuning intensity ($\beta = 2048$).

Benchmark	Number batches per epoch for fine-tuning							
	20		40		100		whole dataset	
	T512	L512	T512	L512	T512	L512	T512	L512
Median Camphor-Menthol	80	57	117	72	126	81	127	80
Median Tadalafil-Sildenafil	24	18	39	17	13	16	21	18
Perindopril Rings	20	17	18	18	17	18	16	15
Zaleplon MPO	32	27	33	25	35	28	44	39
Total number	156	119	207	132	191	143	208	152